

แฮช

Hashing

แฮชทำไม?

- เพื่อเข้าถึงข้อมูลได้อย่างรวดเร็ว
- เราเคยใช้มาก่อนคือ **dictionary**
 - บอก **key** ได้ **value**

Key Value	John 0143	Jack 0001	Jim 0020	Jones 0989	Jane 1233	June 1234	July 9087	Joly 8765
--------------	--------------	--------------	-------------	---------------	--------------	--------------	--------------	--------------

ลองหาแบบ **List**

แฮชทำไม?

- เพื่อเข้าถึงข้อมูลได้อย่างรวดเร็ว
- เราเคยใช้มาก่อนคือ **dictionary**
 - บอก **key** ได้ **value**



Key	John	Jack	Jim	Jones	Jane	June	July	Joly
Value	0143	0001	0020	0989	1233	1234	9087	8765

July → Hash function → 6

ตารางเลข

- เก็บอยู่ในรูปของตารางแบ่งเป็น “ช่อง” (slot)

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

- ตารางเริ่มต้น

[illegible]

ตัวอย่าง

- สนใจเฉพาะตัวเลข
 - 54, 26, 93, 17, 77, 31
- ตารางจำนวน 11 ช่อง
- หากการจับคู่ (map) ระหว่างตัวเลขกับตำแหน่งช่อง
 - ง่ายสุดคือ mod 11
 - $h(\text{item}) = \text{item} \% 11$
 - ฟังก์ชันแฮช

Item	Hash Value
54	10
26	4
93	5
17	6
77	0
31	9

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

ค่า load factor

- $\lambda = \frac{\text{number of items}}{\text{table size}}$
- จากตัวอย่างคือ $\frac{6}{11}$

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

การเข้าถึงข้อมูล

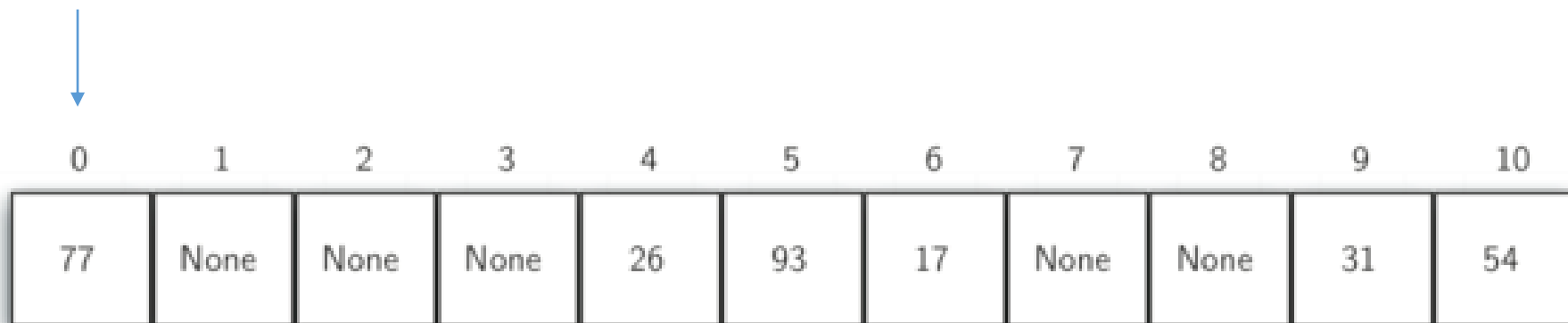
- ใช้ฟังก์ชันแฮช
- เช่น อยากถามว่ามี 54 หรือไม่
 - $h(\text{item}) = \text{item} \% 11$
 - $h(54) = 54 \% 11 = 10$
- $O(1)$



0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

การเข้าถึงข้อมูล

- เหมือนจะดีถ้าข้อมูลทั้งหมดลงที่ช่องต่างกันหมด
- ถ้ามี 44 เพิ่มเข้ามา
 - $h(\text{item}) = \text{item} \% 11$
 - $h(44) = 44 \% 11 = 0$
- เกิดการชน (collision)



0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

ฟังก์ชันแฮช

- เราอยากได้ฟังก์ชันแฮชที่ไม่มีการชน
 - 1 ช่อง 1 ข้อมูล
- ตัวอย่าง ID 3 หลัก
 - ความเป็นไปได้ทั้งหมดคือ 10^3
 - สร้างตารางขนาด 10^3 จะไม่มีการชนเกิดขึ้น
 - แต่ถ้ามีการใช้ ID จริงๆ แค่ 10 ?
 - Load factor ควรจะเยอะด้วย

ฟังก์ชันแฮชที่ดี

- ชนน้อย
- คำนวณง่าย
- กระจายตัว

วิธีการพับ (folding)

- แบ่งเลขเป็นช่วงๆ บวกกันแล้ว mod
- ตัวอย่างหมายเลขโทรศัพท์ 436-555-4601
- แบ่งช่วงละ 2 ได้ (43,65,55,46,01)
- บวกกัน $43+65+55+46+01 = 210$
- mod จำนวนช่อง $210 \% 11 = 1$
- จับใส่ช่อง 1

ตัวกลางยกกำลัง (mid square method)

- $44^2 = 1936$
- เอาสองตัวกลางมา mod จำนวนช่อง $93 \% 11 = 5$

Item	Remainder	Mid-Square
54	10	3
26	4	7
93	5	9
17	6	8
77	0	4
31	9	6

ถ้าเป็นตัวอักษร?

- แปลงตัวอักษรเป็นตัวเลข
- cat
- `ord('c') = 99`
- `ord('a') = 97`
- `ord('t') = 116`
- บวกกัน $99 + 97 + 116 = 312$
- mod ด้วยจำนวนช่อง $312 \% 11$

```
def hash(astring, tablesize):  
    sum = 0  
    for pos in range(len(astring)):  
        sum = sum + ord(astring[pos])  
  
    return sum%tablesize
```

Anagram

- ถ้าเป็น anagram จะได้ค่าแฮชเดียวกัน
- $\text{hash}(\text{'cat'}) = \text{hash}(\text{'act'})$ ขน
- ใส่ค่าถ่วงน้ำหนักตามตำแหน่ง
- $\text{hash}(\text{'cat'}) = 99 * 1 + 97 * 2 + 116 * 3 = 641 \% 11 = 3$
- $\text{hast}(\text{'act'}) = 97 * 1 + 99 * 2 + 116 * 3 = 643 \% 11 = 5$

mod ด้วยจำนวนช่องเสมอ !!!!

แก้ปัญหาการชน

ไล่หาช่องว่างช่องแรกที่เราเจอ (linear probing)

- เพิ่ม 44
- $\text{hash}(44) = 44 \% 11 = 0$ ชน

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

- ช่องว่างถัดไปคือ 1

0	1	2	3	4	5	6	7	8	9	10
77	44	None	None	26	93	17	None	None	31	54

ไต่หาช่องว่างช่องแรกที่เราเจอ (linear probing)

- เพิ่ม 55
- $\text{hash}(55) = 55 \% 11 = 0$ ชน

0	1	2	3	4	5	6	7	8	9	10
77	44	None	None	26	93	17	None	None	31	54

- ช่องว่างถัดไปคือ 2

0	1	2	3	4	5	6	7	8	9	10
77	44	55	None	26	93	17	None	None	31	54

ไต่หาช่องว่างช่องแรกที่เจอ (linear probing)

- เพิ่ม 20
- $\text{hash}(20) = 20 \% 11 = 9$ ชน

0	1	2	3	4	5	6	7	8	9	10
77	44	55	None	26	93	17	None	None	31	54

- ช่องว่างถัดไปคือ 3 (หมดตารางวนกลับมาเริ่มที่ 0)

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

การเข้าถึงข้อมูลแบบ linear probing

- หาเจอตอบเลยว่า **True**
 - หา 93
 - $\text{hash}(93) = 93 \% 11 = 5$ เจอ
- หาไม่เจอยังตอบไม่ได้
 - หา 20
 - $\text{hash}(20) = 20 \% 11 = 9$ ชน
 - วิ่งหา 20 เจอตอบ **True** วิ่งเจอช่องว่าง (**None**) ถึงจะตอบ **False**

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

ปัญหาของ linear probing

- ข้อมูลที่ชนกันด้วยค่าแฮชเดียวกันจะกระจุกตัว
- ตัวที่ $\text{mod } 11 = 0$ กระจุกตัวกัน
- เต็ม 20 ต้องไล่หลายช่องกว่าจะเจอช่องว่าง

0	1	2	3	4	5	6	7	8	9	10
77	44	55	None	26	93	17	None	None	31	54

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

กระโดดข้าม

- กระโดดหาช่องว่างทีละ 3 ช่อง
- เต็ม 44 คนที่ช่อง 0 กระโดดไป 3 เจอช่อง 3 ว่าง
- เต็ม 55 คนที่ช่อง 0 กระโดดไป 3 เจอช่อง 3 คนอีก กระโดดไป 3 ช่องเจอช่อง 6 คนอีก กระโดดไป 3 ช่องเจอช่อง 9 คนอีก กระโดดไป 3 เจอช่อง 1 ว่าง
- เต็ม 20 ?

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

0	1	2	3	4	5	6	7	8	9	10
77	55	None	44	26	93	17	20	None	31	54

การแฮช (rehash)

- การหาช่องว่างเมื่อเกิดการชน
- ฟังก์ชันการแฮช
- $\text{rehash}(\text{pos}) = (\text{pos} + 1) \% \text{sizeof table}$
- $\text{rehash}(\text{pos}) = (\text{pos} + 3) \% \text{sizeof table}$
- $\text{rehash}(\text{pos}) = (\text{pos} + \text{skip}) \% \text{sizeof table}$
- วิธีการกระโดดข้ามควรใช้กับขนาดตารางที่เป็นจำนวนเฉพาะ

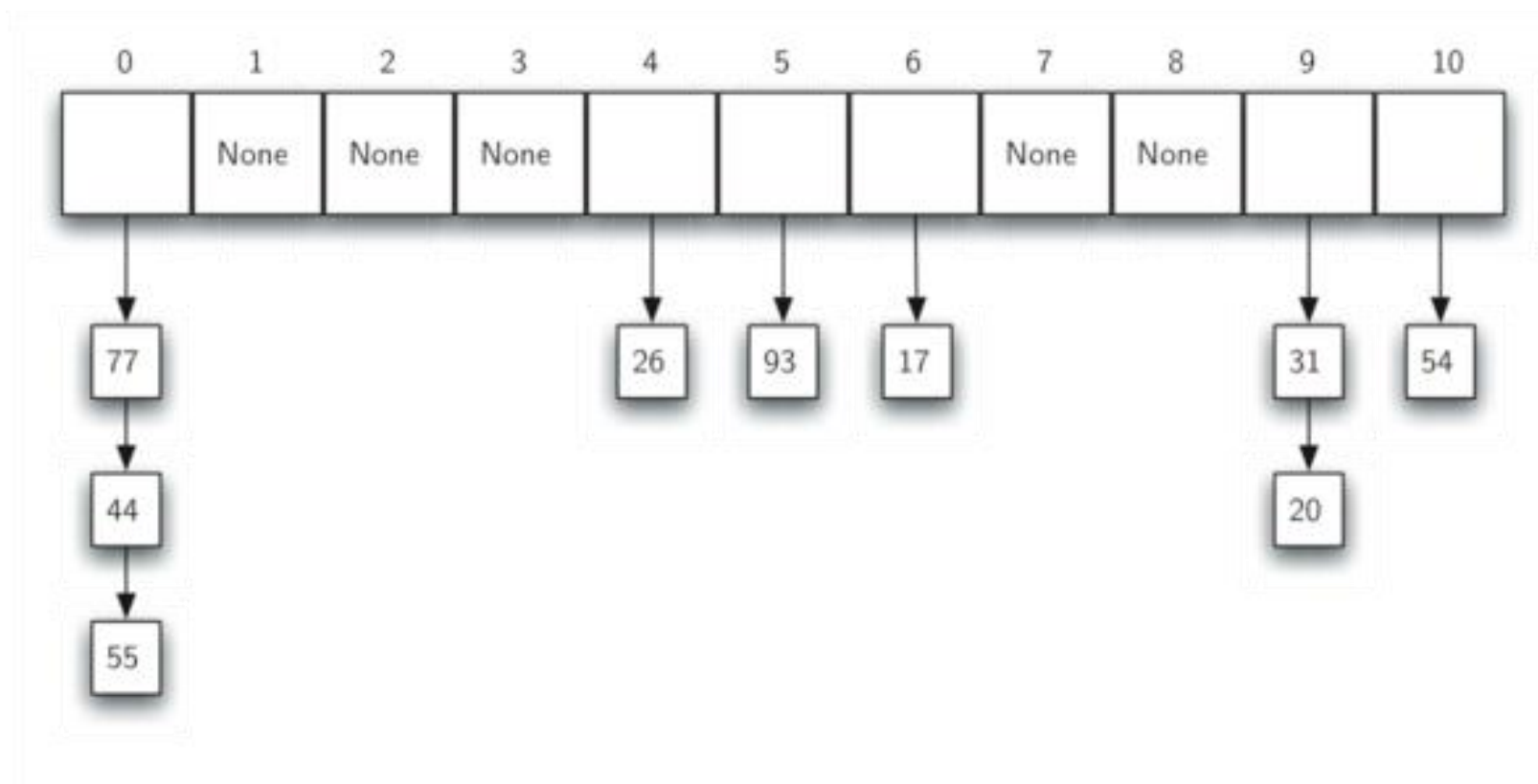
กระโดดแบบยกกำลังสอง

- ชนครั้งแรกกระโดดจากค่าแฮชเดิม $1^2 = 1$
- ชนครั้งที่สองกระโดดจากค่าแฮชเดิม $2^2 = 4$
- ชนครั้งที่สองกระโดดจากค่าแฮชเดิม $3^2 = 9$
- $\text{rehash}(h) = h + 1$
- $\text{rehash}(h) = h + 4$
- $\text{rehash}(h) = h + 9$

0	1	2	3	4	5	6	7	8	9	10
77	None	None	None	26	93	17	None	None	31	54

0	1	2	3	4	5	6	7	8	9	10
77	44	20	55	26	93	17	None	None	31	54

การเก็บเป็นลูกโซ่



Map ADT

- หลักการเดียวกับ **dictionary**
 - เรามองว่า **key** คือค่าที่จะนำไปคำนวณแฮชฟังก์ชันแล้วเก็บ **val** คู่ไปด้วย เรียกอีกอย่างว่า **Map**
- **Map()** สร้าง **map** ว่างแล้วคืนค่า **map** ว่างนั้น
- **put(key, val)** เพิ่มคู่ **key-val** เข้าใน **map** ถ้า **key** มีอยู่แล้วใน **map** ให้แทนที่ค่าเก่าด้วย **val** ใหม่
- **get(key)** คืนค่าที่ **key** นั้น หรือ **None** ถ้าไม่มี **key** นั้น
- **del map[key]** ลบค่าที่ **key**
- **len()** คืนค่าจำนวนที่เก็บอยู่ใน **map**.
- **key in map** คืนค่า **True** เมื่อมี **key** ใน **map** หรือ **False** เมื่อไม่มี **key** นั้น

สร้าง class HashTable

```
class HashTable:  
    def __init__(self):  
        self.size = 11  
        self.slots = [None] * self.size  
        self.data = [None] * self.size
```

```

def put(self, key, data):
    hashvalue = self.hashfunction(key, len(self.slots))

    if self.slots[hashvalue] == None:
        self.slots[hashvalue] = key
        self.data[hashvalue] = data
    else:
        if self.slots[hashvalue] == key:
            self.data[hashvalue] = data #replace
        else:
            nextslot = self.rehash(hashvalue, len(self.slots))
            while self.slots[nextslot] != None and \
                  self.slots[nextslot] != key:
                nextslot = self.rehash(nextslot, len(self.slots))

            if self.slots[nextslot] == None:
                self.slots[nextslot] = key
                self.data[nextslot] = data
            else:
                self.data[nextslot] = data #replace

def hashfunction(self, key, size):
    return key % size

def rehash(self, oldhash, size):
    return (oldhash + 1) % size

```

```

def get(self, key):
    startslot = self.hashfunction(key, len(self.slots))

    data = None
    stop = False
    found = False
    position = startslot
    while self.slots[position] != None and \
           not found and not stop:
        if self.slots[position] == key:
            found = True
            data = self.data[position]
        else:
            position = self.rehash(position, len(self.slots))
            if position == startslot:
                stop = True
    return data

def __getitem__(self, key):
    return self.get(key)

def __setitem__(self, key, data):
    self.put(key, data)

```

ใช้งาน HashTable

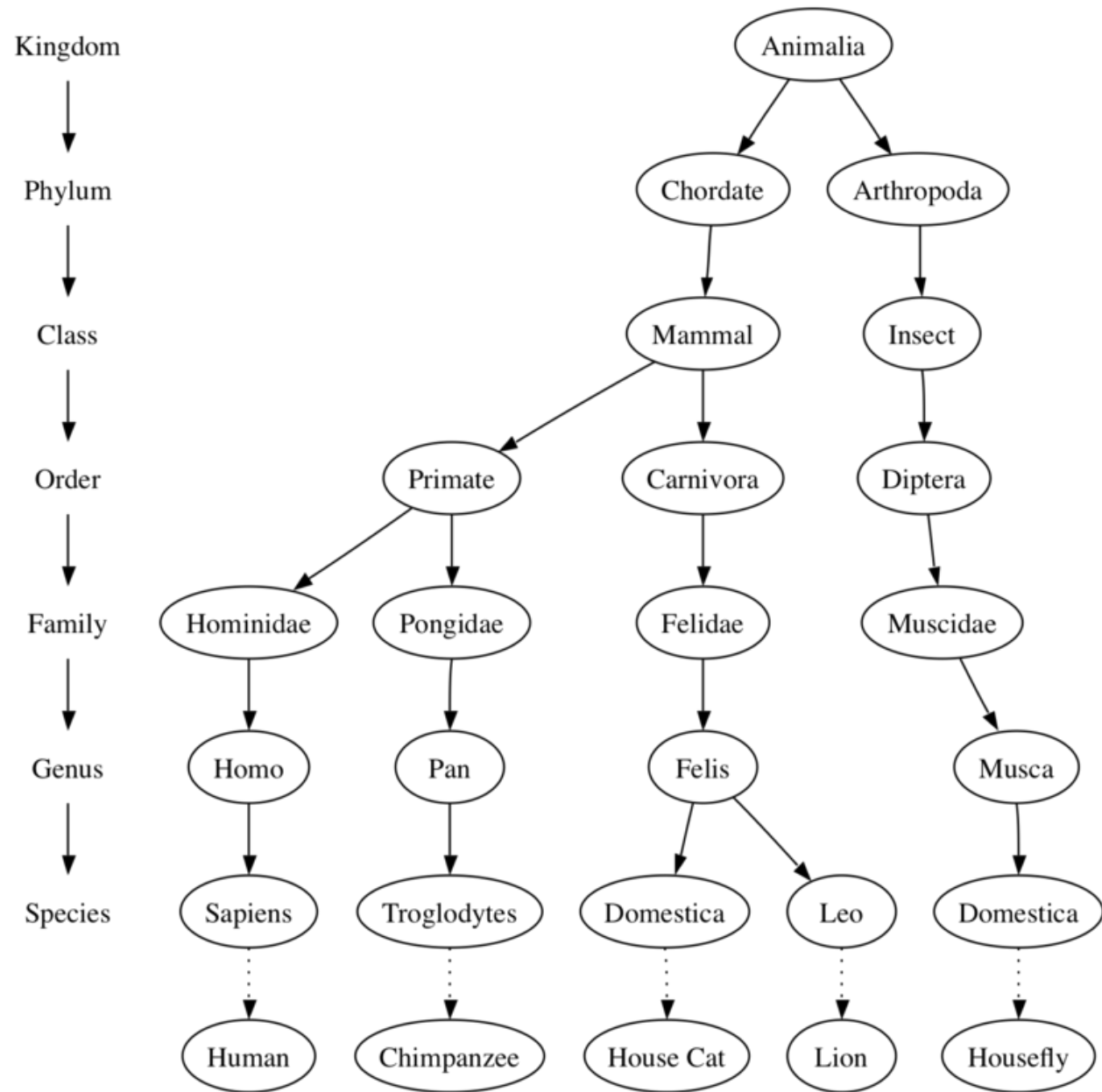
```
>>> H=HashTable()
>>> H[54]="cat"
>>> H[26]="dog"
>>> H[93]="lion"
>>> H[17]="tiger"
>>> H[77]="bird"
>>> H[31]="cow"
>>> H[44]="goat"
>>> H[55]="pig"
>>> H[20]="chicken"
>>> H.slots
[77, 44, 55, 20, 26, 93, 17, None, None, 31, 54]
>>> H.data
['bird', 'goat', 'pig', 'chicken', 'dog', 'lion',
 'tiger', None, None, 'cow', 'cat']
```

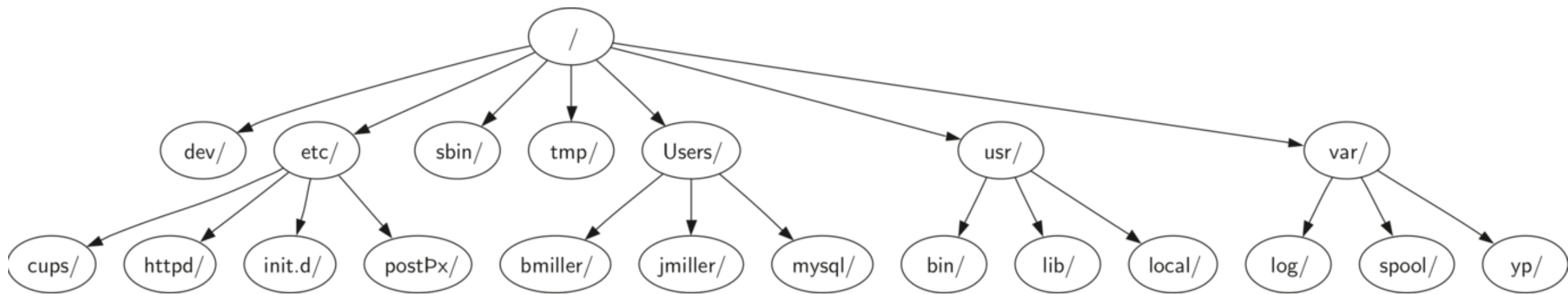
```
>>> H[20]
'chicken'
>>> H[17]
'tiger'
>>> H[20]='duck'
>>> H[20]
'duck'
>>> H.data
['bird', 'goat', 'pig', 'duck', 'dog', 'lion',
 'tiger', None, None, 'cow', 'cat']
>> print(H[99])
None
```

ต้นไม้ (tree)

เรียนเพื่ออะไร?

- เข้าใจหลักการของโครงสร้างข้อมูลต้นไม้
- เพื่อให้เห็นว่าต้นไม้สามารถใช้ในการสร้างโครงสร้างข้อมูลแมพได้
- เพื่อสร้างต้นไม้โดยใช้ **list**
- เพื่อสร้าง **class** ของต้นไม้
- เพื่อสร้างต้นไม้ในรูปแบบความสัมพันธ์เวียนเกิด
- เพื่อสร้างคิวแบบมีความสำคัญ (**priority queue**) ด้วยฮีพ (**heap**)





```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
  xml:lang="en" lang="en">
```

```
<head>
```

```
  <meta http-equiv="Content-Type"
```

```
    content="text/html; charset=utf-8" />
```

```
  <title>simple</title>
```

```
</head>
```

```
<body>
```

```
<h1>A simple web page</h1>
```

```
<ul>
```

```
  <li>List item one</li>
```

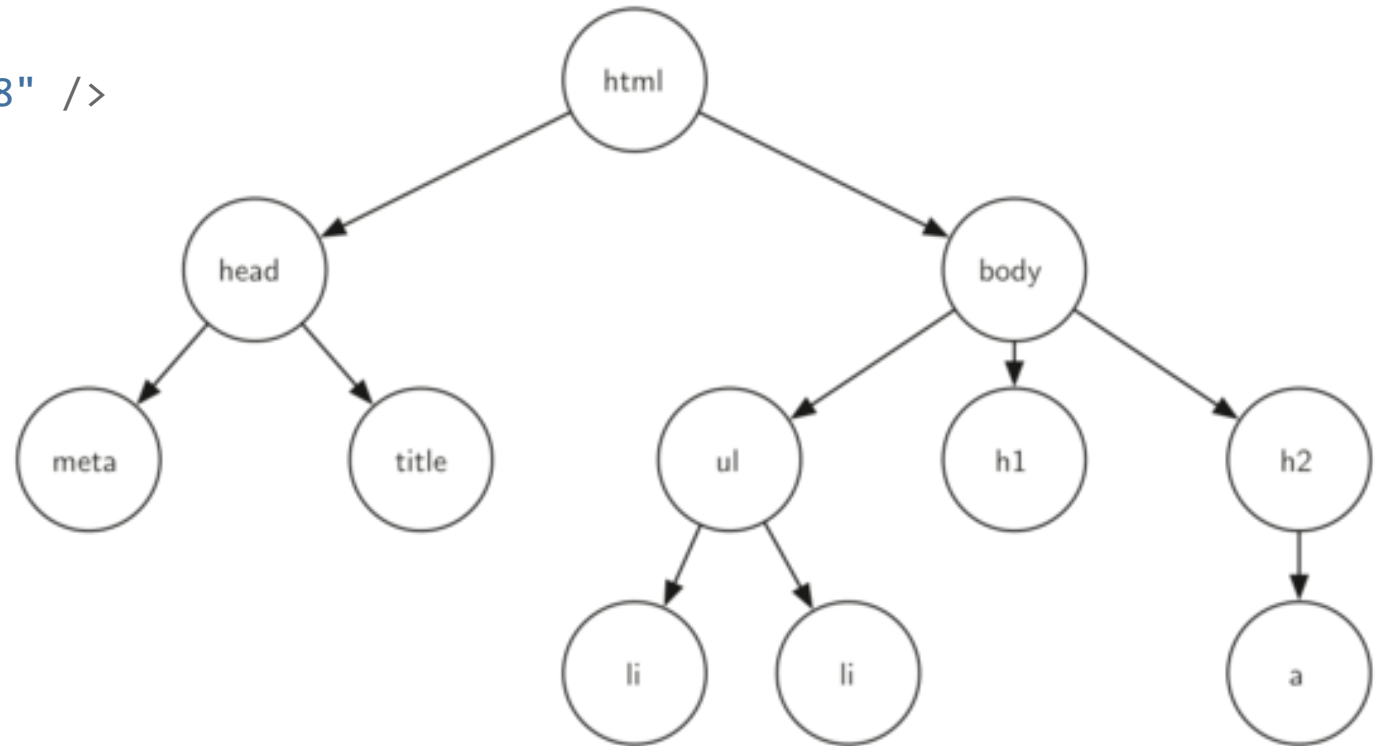
```
  <li>List item two</li>
```

```
</ul>
```

```
<h2><a href="http://www.cs.luther.edu">Luther CS </a></h2>
```

```
</body>
```

```
</html>
```

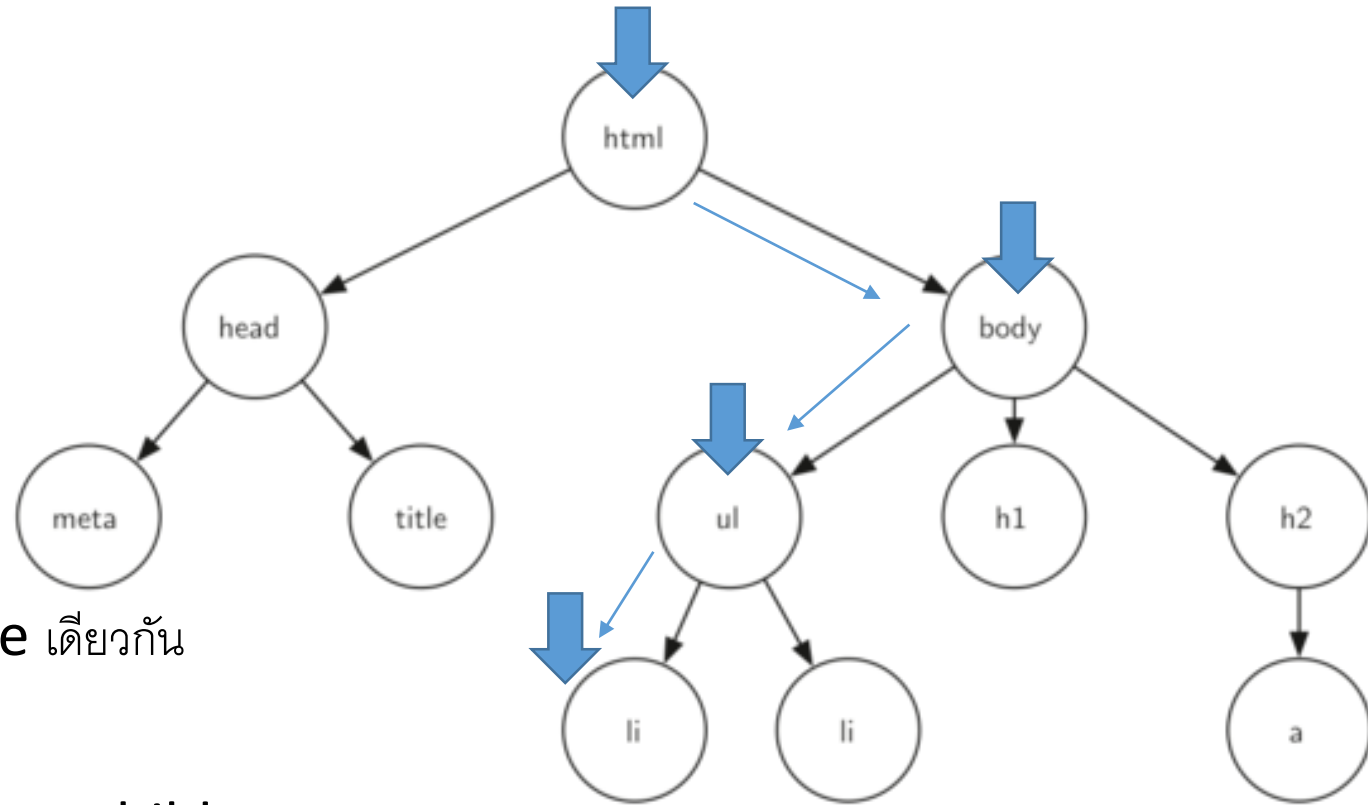


คำศัพท์

- ปม (node)
 - มีชื่อได้ เรียกว่า **key**
 - มีข้อมูลอื่นๆ เพิ่มได้ เรียกว่า **payload**
- เส้นเชื่อม (edge)
 - เชื่อมระหว่าง **2 node** เพื่อแสดงความสัมพันธ์ระหว่างกัน
 - ทุก **node** ต้องมีอย่างน้อย **1 edge** เข้าหามัน (ยกเว้น **root**)
 - **1 node** สามารถมีได้มากกว่า **1 edge** ออกจากตัวมัน
- ราก (root)
 - เป็น **node** ที่มีแต่ขาออก

คำศัพท์

- เส้นทาง (path)
 - list ของลำดับของ node ที่เชื่อมด้วย edge
- ลูก (children)
 - set ของ node ที่มี edge ขาเข้าจาก node เดียวกัน
- พ่อ (parent)
 - node ที่เป็นขาออกของ edge เพื่อไปเชื่อมต่อกับ children
- พี่น้อง (sibling)
 - node ที่มี parent เดียวกัน

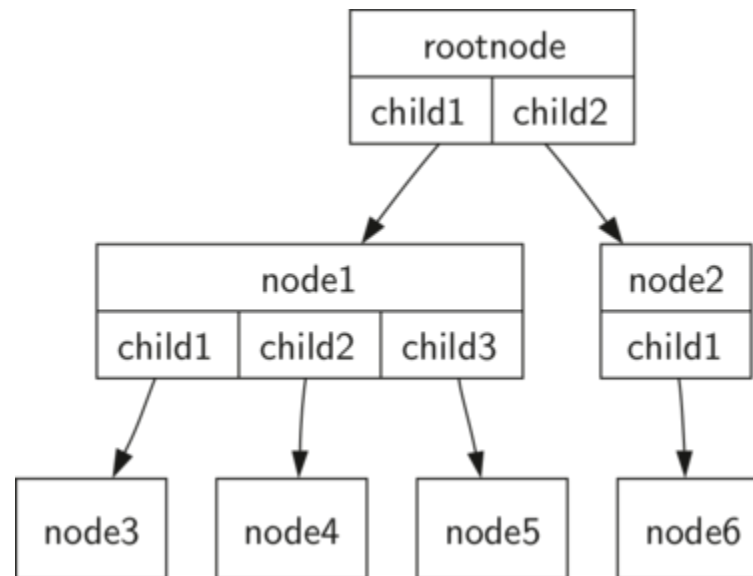


คำศัพท์

- ต้นไม้ย่อย (subtree)
 - set ของ node และ edge ประกอบด้วย parent และผู้สืบสายพันธุ์
- ปมใบไม้ (leaf node)
 - node ที่ไม่มี children
- ระดับ (level)
 - จำนวนของ edge จาก root ถึงตัวมัน
- ความสูง (height)
 - level ที่มากที่สุดของ node ในต้นไม้

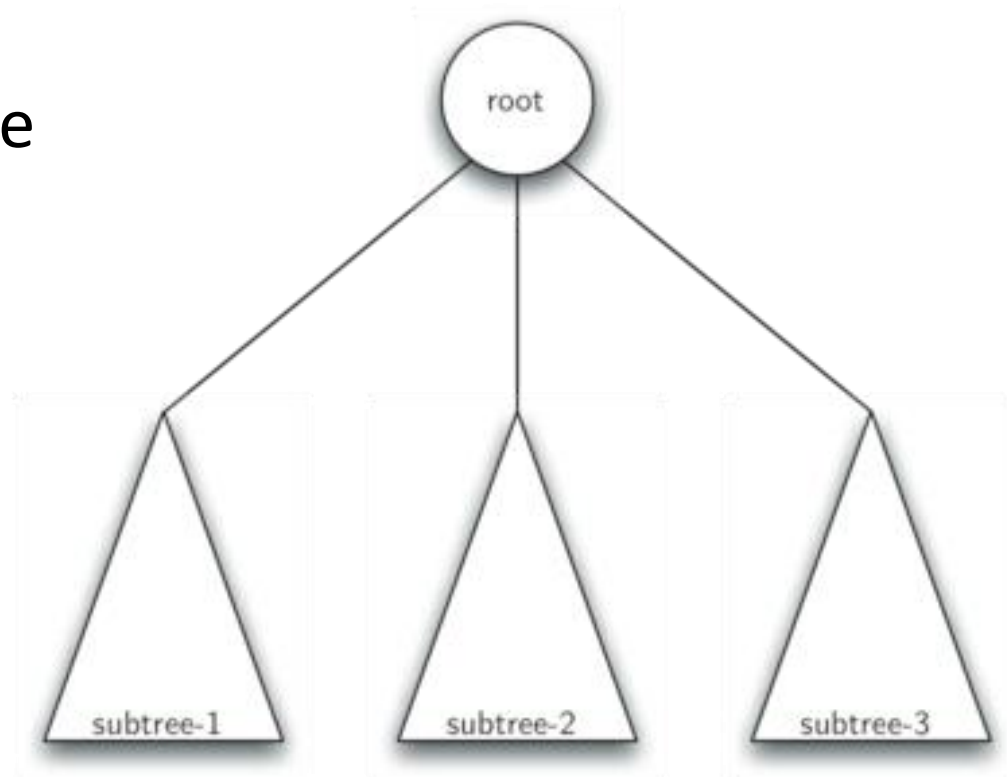
นิยาม 1

- **tree** ประกอบด้วย **set** ของ **node** และ **edge** ซึ่งเชื่อมคู่ของ **node** ซึ่ง **tree** มีคุณสมบัติดังนี้
 - ต้องมี **1 node** เป็น **root**
 - ทุก **node n** ยกเว้น **root** ต้องมี **edge** เชื่อมกับ **node p** ที่เป็น **parent** ของมัน
 - มีแค่ **path** เดียวจาก **root** ถึงแต่ละ **node**
 - ถ้าทุก **node** มีจำนวน **children** ไม่เกิน **2** จะเรียกว่า ต้นไม้ทวิภาค (**binary tree**)



นิยาม 2

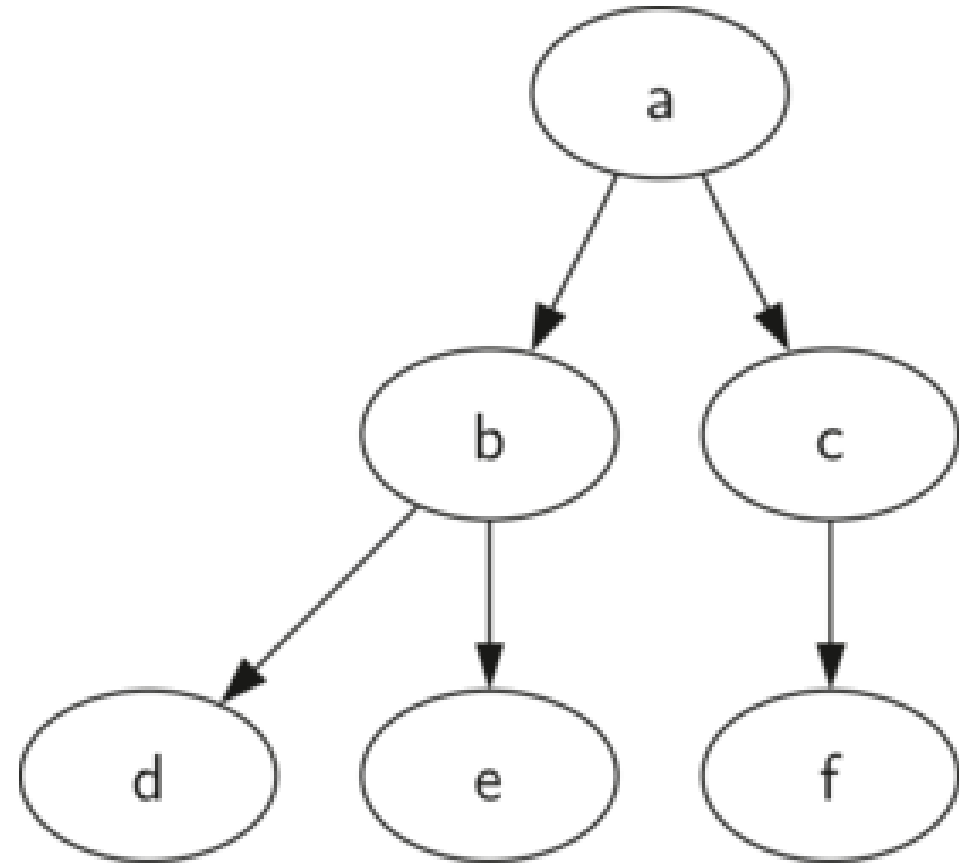
- **tree** จะว่างหรือประกอบด้วย **root** ที่มีหรือไม่มี **subtree** ก็ถือว่าเป็น **tree**
- **root** ของแต่ละ **subtree** ที่เชื่อมต่อกับ **parent** ของ **subtree** ด้วย **edge**
 - เป็นความสัมพันธ์แบบเวียนเกิด
- **tree** ในรูปมีอย่างน้อย 4 node



สร้าง tree ด้วย list ของ list

- แต่ละ node คือ list
 - เก็บค่าไว้ที่ item แรกของ list
 - item ที่ 2 เก็บ list ของ subtree ซ้าย
 - item ที่ 3 เก็บ list ของ subtree ขวา

```
myTree = ['a',  #root
          ['b',  #left subtree
            ['d', [], []],
            ['e', [], []] ],
          ['c',  #right subtree
            ['f', [], []],
            [] ]
        ]
```



การใช้งาน

- `myTree[0]` คือ ค่าที่ **root**
- `myTree[1]` คือ **subtree** ซ้าย
- `myTree[2]` คือ **subtree** ขวา
- เป็นโครงสร้างแบบความสัมพันธ์เวียนเกิด
- ไม่จำกัดว่าจะต้องเป็น **binary tree**

```
myTree = ['a', ['b', ['d',[],[]], ['e',[],[]] ], ['c', ['f',[],[]], [] ]  
print(myTree)  
print('left subtree = ', myTree[1])  
print('root = ', myTree[0])  
print('right subtree = ', myTree[2])
```

```
def BinaryTree(r):  
    return [r, [], []]  
  
def getRootVal(root):  
    return root[0]  
  
def setRootVal(root,newVal):  
    root[0] = newVal  
  
def getLeftChild(root):  
    return root[1]  
  
def getRightChild(root):  
    return root[2]
```

```
def insertLeft(root,newBranch):  
    t = root.pop(1)  
    if len(t) > 1:  
        root.insert(1,[newBranch,t,[]])  
    else:  
        root.insert(1,[newBranch, [], []])  
    return root  
  
def insertRight(root,newBranch):  
    t = root.pop(2)  
    if len(t) > 1:  
        root.insert(2,[newBranch,[],t])  
    else:  
        root.insert(2,[newBranch,[],[]])  
    return root
```

```
r = BinaryTree(3)
insertLeft(r,4)
insertLeft(r,5)
insertRight(r,6)
insertRight(r,7)
l = getLeftChild(r)
print(l)
```

```
setRootVal(l,9)
print(r)
insertLeft(l,11)
print(r)
print(getRightChild(getRightChild(r)))
```

```
[5, [4, [], []], []]
[3, [9, [4, [], []], []], [7, [], [6, [], []]]]
[3, [9, [11, [4, [], []], []], []], [7, [], [6,
[], []]]]
[6, [], []]
```

ฮีพทวิภาค

Binary heap

คิวแบบมีลำดับความสำคัญ (priority queue)

- queue ปกติจะมีลำดับตามการเข้า
- priority queue มีการปรับลำดับตามความสำคัญ
 - ความสำคัญสูงอยู่ที่หัว queue
 - ความสำคัญต่ำอยู่ที่ปลาย queue
- เมื่อมีการ enqueue item จะถูกจัดไปอยู่ในลำดับตามความสำคัญ
- ง่ายสุดในการสร้างคือ insert ด้วย $O(n)$ แล้วจัดลำดับความสำคัญด้วยการ sort ใช้เวลา $O(n \log n)$
- ดีกว่านั้นคือใช้ ฮีพทวิภาค
 - enqueue ใช้ $O(\log n)$
 - dequeue ใช้ $O(\log n)$

ฮีพทวิภาค

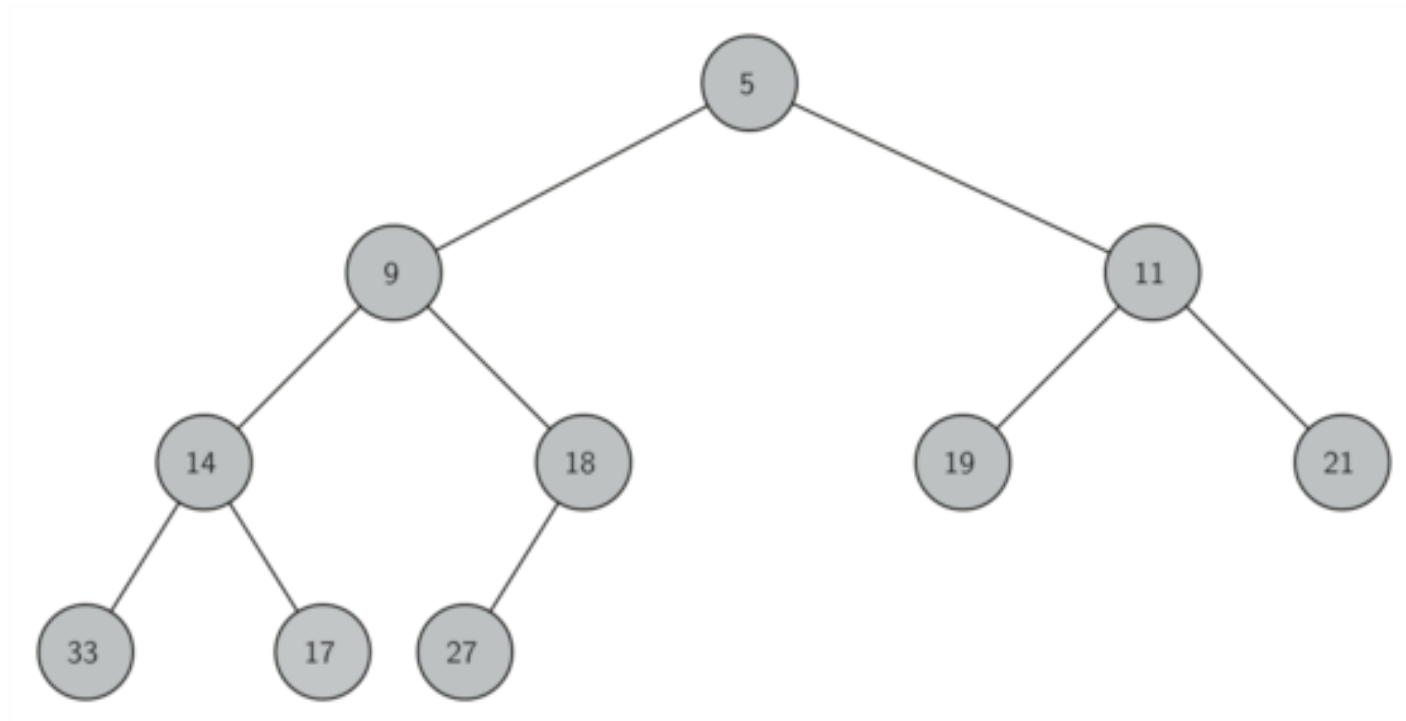
- มีความคล้ายกับต้นไม้
- การสร้างจริงๆ ใช้แค่ **list**
- มี 2 ประเภทคือ
 - **max heap** ความสำคัญสูงสุดคือมีค่า **key** มากสุด
 - **min heap** ความสำคัญสูงสุดคือมีค่า **key** น้อยสุด

method ของฮีพทวิภาค

- **BinaryHeap()** สร้างฮีพว่าง
- **insert(k)** เพิ่ม **item** ใน ฮีพ
- **findMin()** คืนค่า **item** ที่มีค่า **key** น้อยที่สุด
- **delMin()** คืนค่า **item** ที่มีค่า **key** น้อยที่สุดแล้วลบออกด้วย
- **isEmpty()** ถามว่าฮีพว่างหรือไม่
- **size()** คืนค่าจำนวน **item** ในฮีพ
- **buildHeap(list)** สร้างฮีพใหม่จาก **list**

โครงสร้าง

- คล้ายต้นไม้ที่รักษาความสมดุล
 - ปม (**node**) ฝั่งซ้ายและฝั่งขวามีจำนวนเท่ากัน
 - ใช้ต้นไม้สมมาตร
 - แต่ละลำดับชั้นมี **node** เต็ม
 - ชั้นสุดท้ายเรียกว่าใบเรียงวางจากซ้ายไปขวา

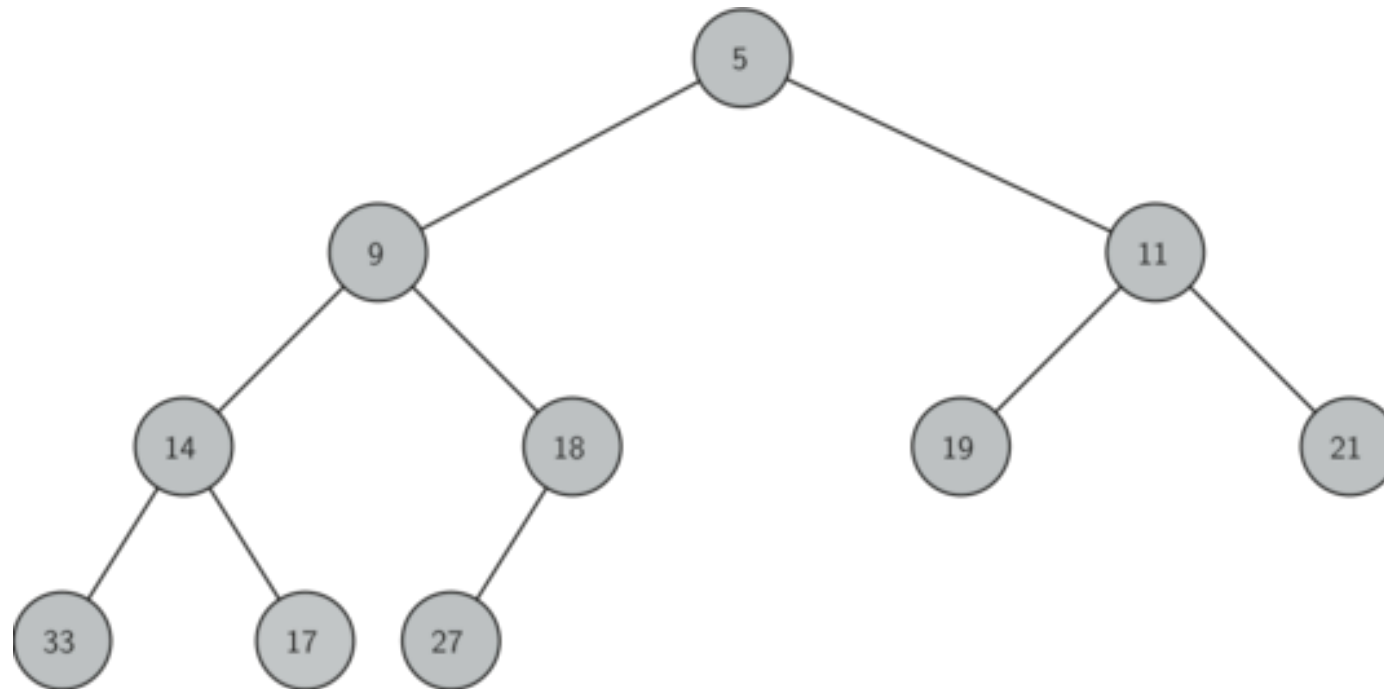


ต้นไม้สมบูรณ์

- สร้างได้ด้วย list
- ที่ parent node p
 - left child อยู่ที่ตำแหน่ง $2p$
 - right child อยู่ที่ตำแหน่ง $2p + 1$
- ที่ node p ใดๆ parent อยู่ที่ $p // 2$

คุณสมบัติของฮีพ

- key ที่ parent มีค่าน้อยกว่าหรือเท่ากับ children เสมอ



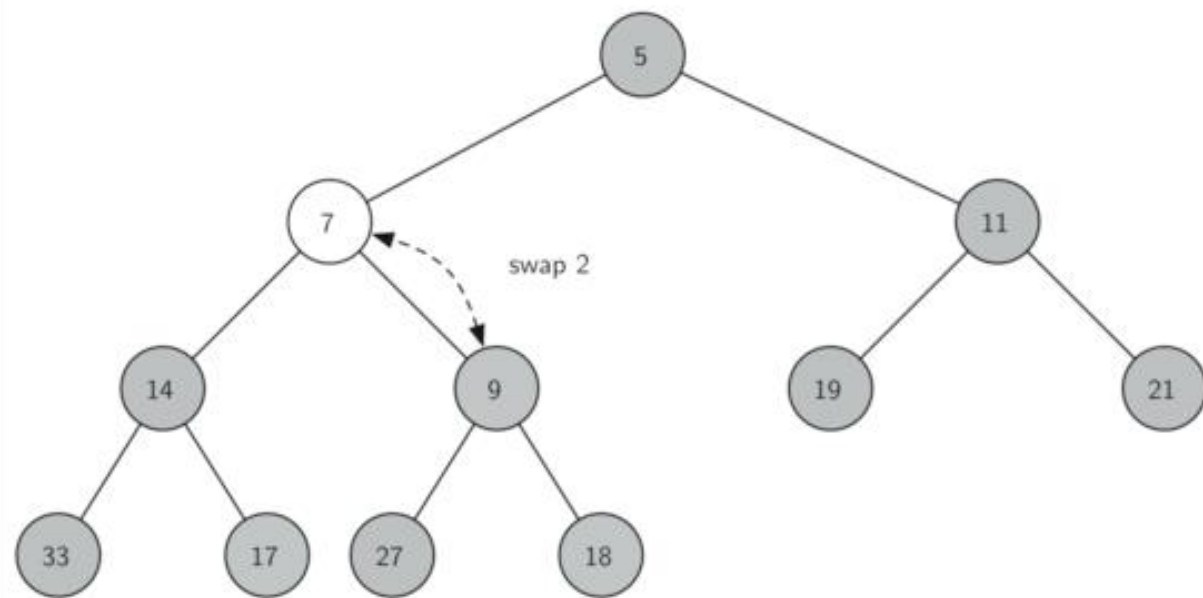
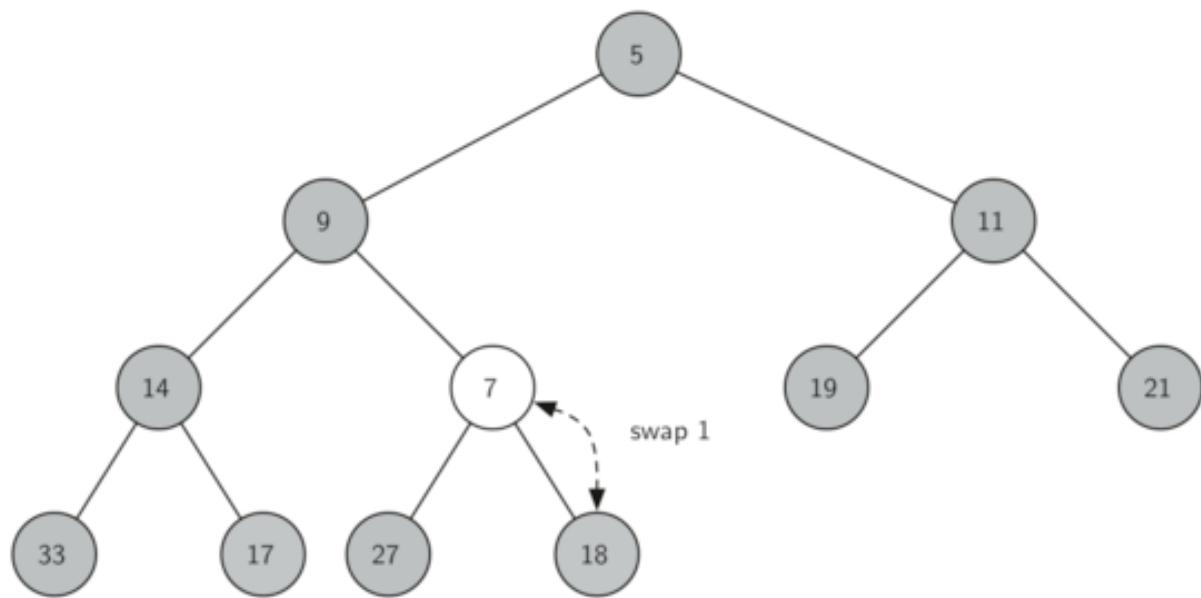
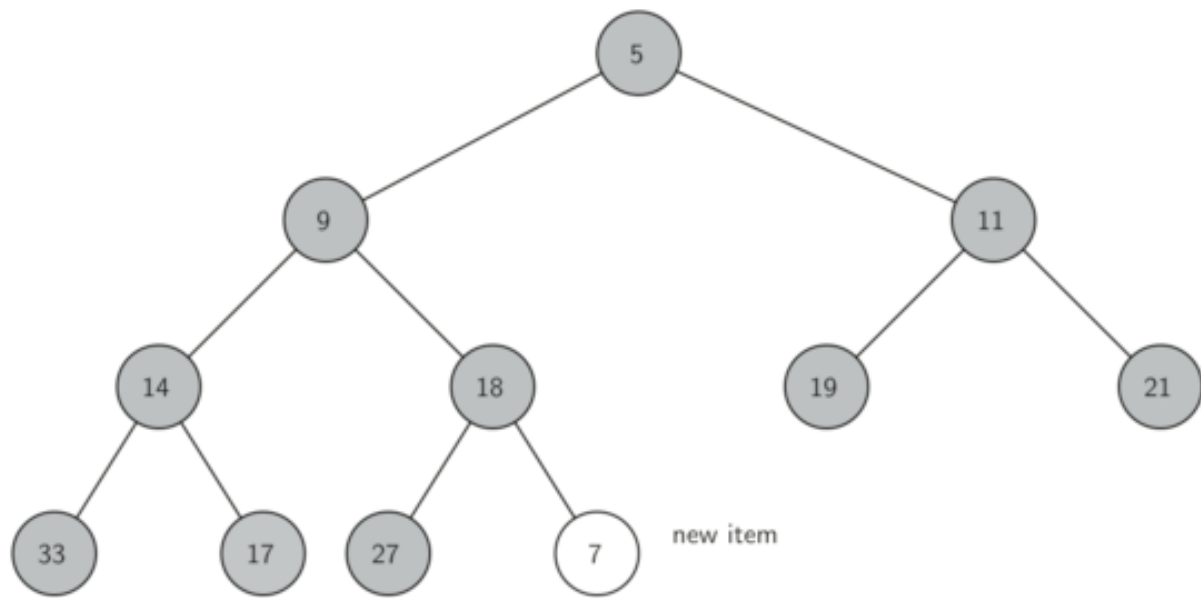
0	5	9	11	14	18	19	21	33	17	27	
0	1	2	3	4	5	6	7	8	9	10	11

การสร้าง class ของฮีพ

```
class BinHeap:  
    def __init__(self):  
        self.heapList = [0]  
        self.currentSize = 0
```

insert

- ทำโดย **append item** เข้าไปใน **list**
- รักษาสภาพความสมบูรณ์เสมอ
- ดัน **item** กลับขึ้นด้านบนเพื่อให้อยู่ในลำดับที่ถูกต้อง (**percolate up**)
 - **parent** มี **key** น้อยกว่าหรือเท่ากับ **children** เสมอ

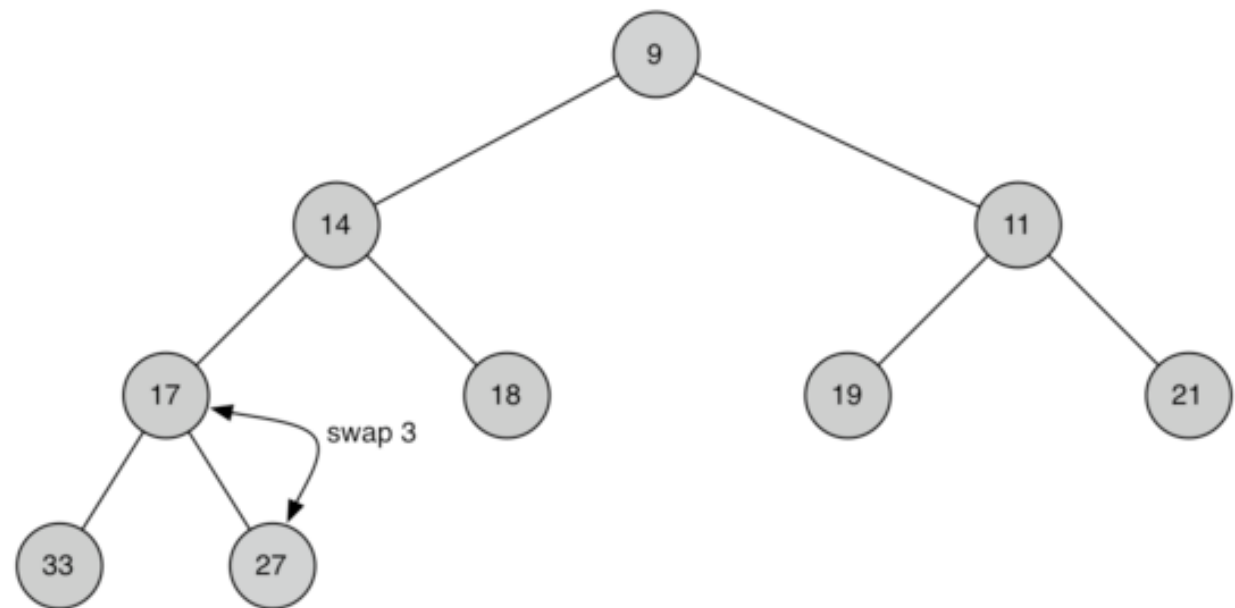
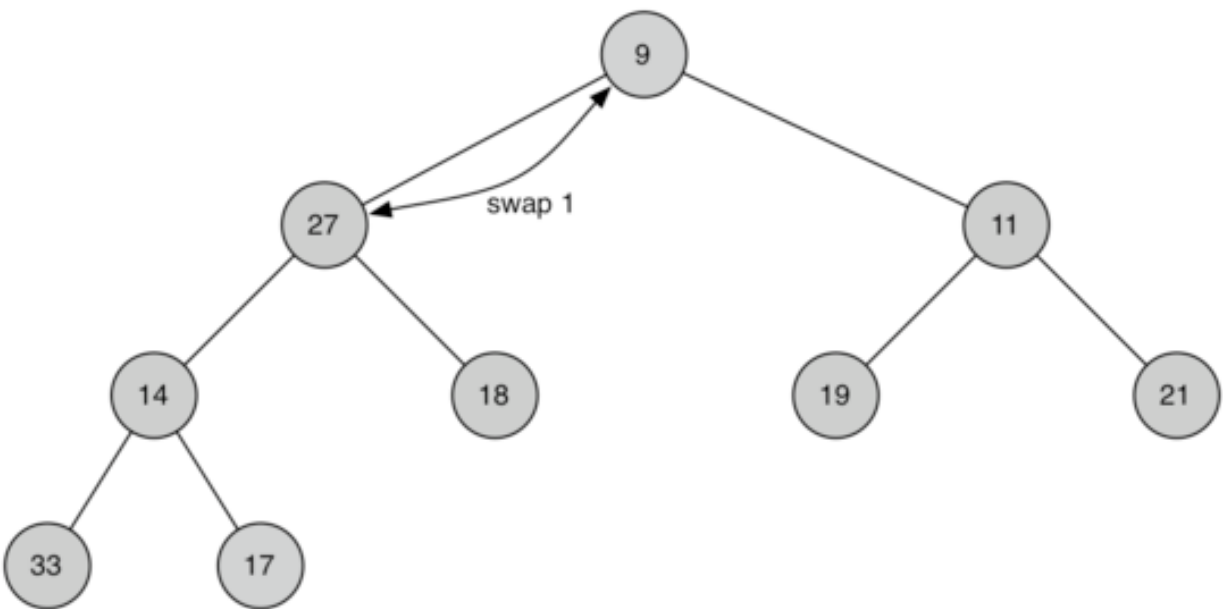
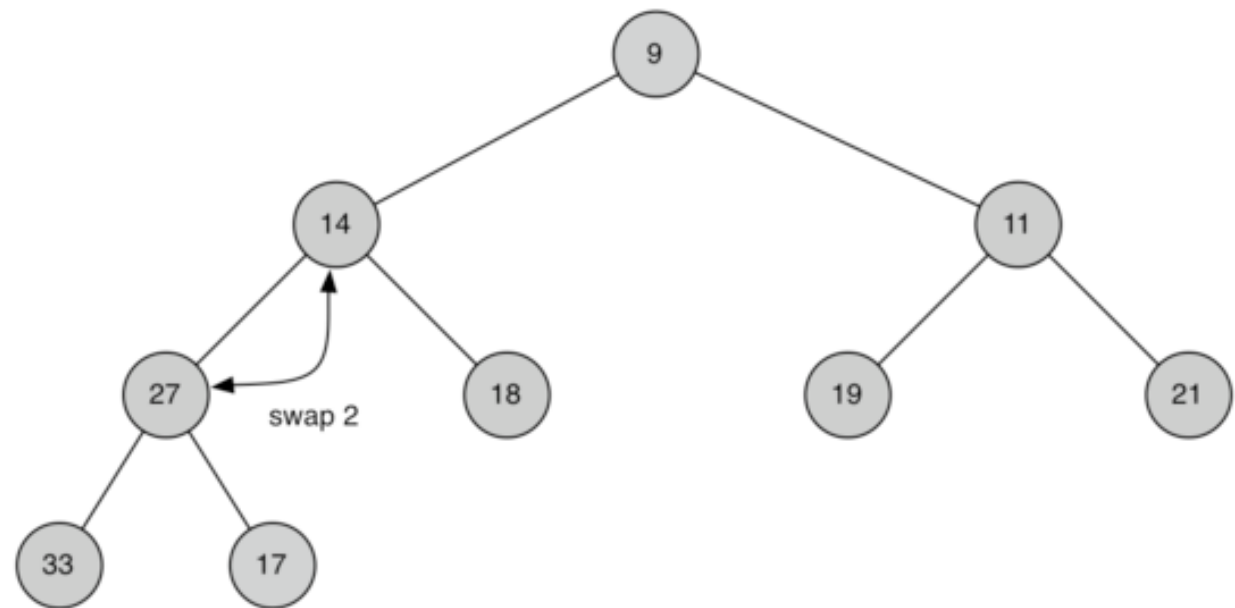
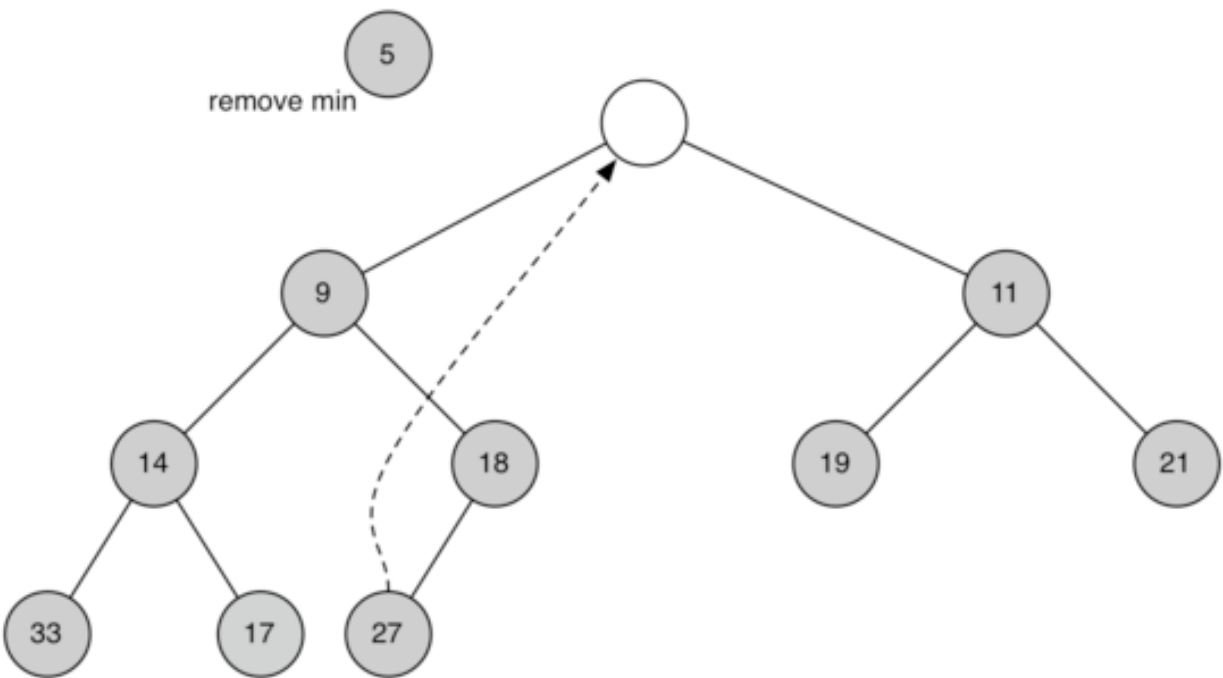


```
def percUp(self,i):  
    while i // 2 > 0:  
        if self.heapList[i] < self.heapList[i // 2]:  
            tmp = self.heapList[i // 2]  
            self.heapList[i // 2] = self.heapList[i]  
            self.heapList[i] = tmp  
        i = i // 2
```

```
def insert(self,k):  
    self.heapList.append(k)  
    self.currentSize = self.currentSize + 1  
    self.percUp(self.currentSize)
```

delMin

- ต้องหา **node** ที่มี **key** น้อยสุดขึ้นมาแทนที่
- เอา **node** สุดท้ายขึ้นมาก่อน
- ไต่กลับลงไปตำแหน่งที่ (**percolate down**)



```
def percDown(self,i):
```

```
    while (i * 2) <= self.currentSize:
```

```
        mc = self.minChild(i)
```

```
        if self.heapList[i] > self.heapList[mc]:
```

```
            tmp = self.heapList[i]
```

```
            self.heapList[i] = self.heapList[mc]
```

```
            self.heapList[mc] = tmp
```

```
    i = mc
```

```
def minChild(self,i):
```

```
    if i * 2 + 1 > self.currentSize:
```

```
        return i * 2
```

```
    else:
```

```
        if self.heapList[i*2] < self.heapList[i*2+1]:
```

```
            return i * 2
```

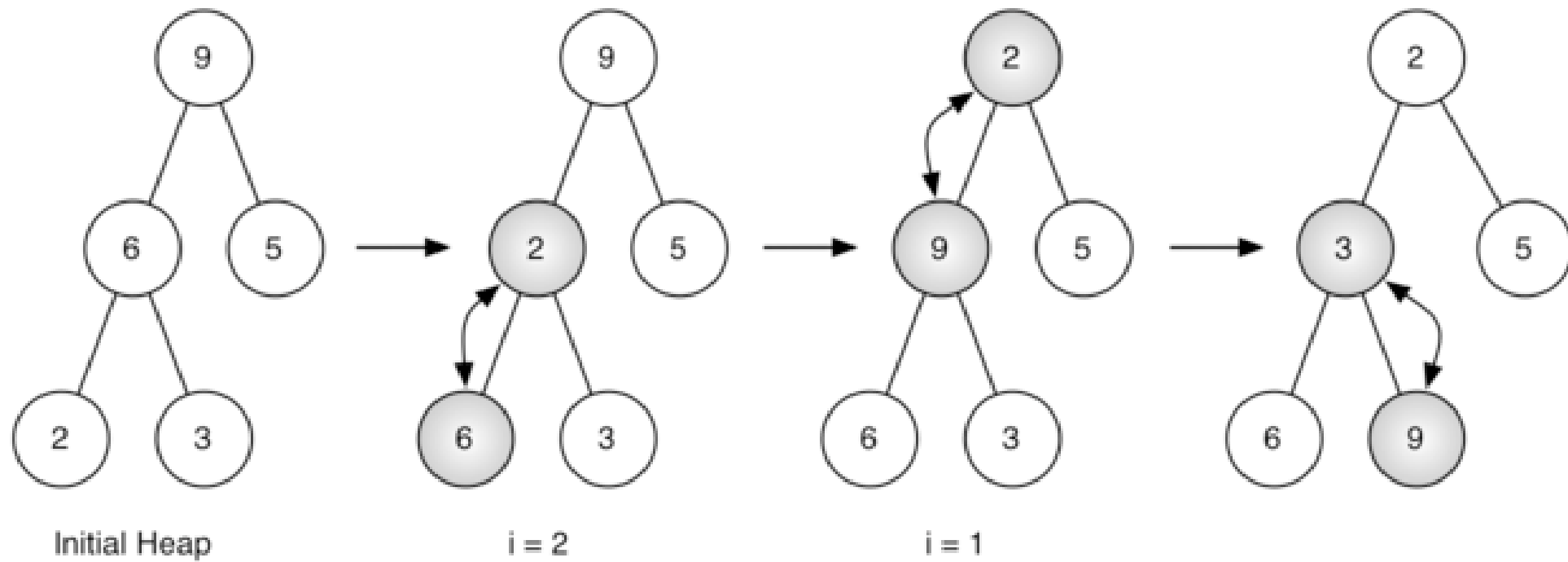
```
        else:
```

```
            return i * 2 + 1
```

```
def delMin(self):  
    retval = self.heapList[1]  
    self.heapList[1] = self.heapList[self.currentSize]  
    self.currentSize = self.currentSize - 1  
    self.heapList.pop()  
    self.percDown(1)  
    return retval
```

สร้างฮีพจาก list

- ง่ายสุดคือค่อยๆ insert ทีละค่าใช้ $O(n \log n)$
- สร้างจาก list ทั้งหมดใช้ $O(n)$
- เริ่มจากครึ่งของ list
 - อีกครึ่งที่เหลือเป็น leaf node ที่ไม่มี child
- ขยับไปข้างหน้า list เรื่อยๆ (วิ่งขึ้นบนของ tree) แล้วไถ่กลับลงมา



$i = 2$ $[0, 9, 5, 6, 2, 3]$

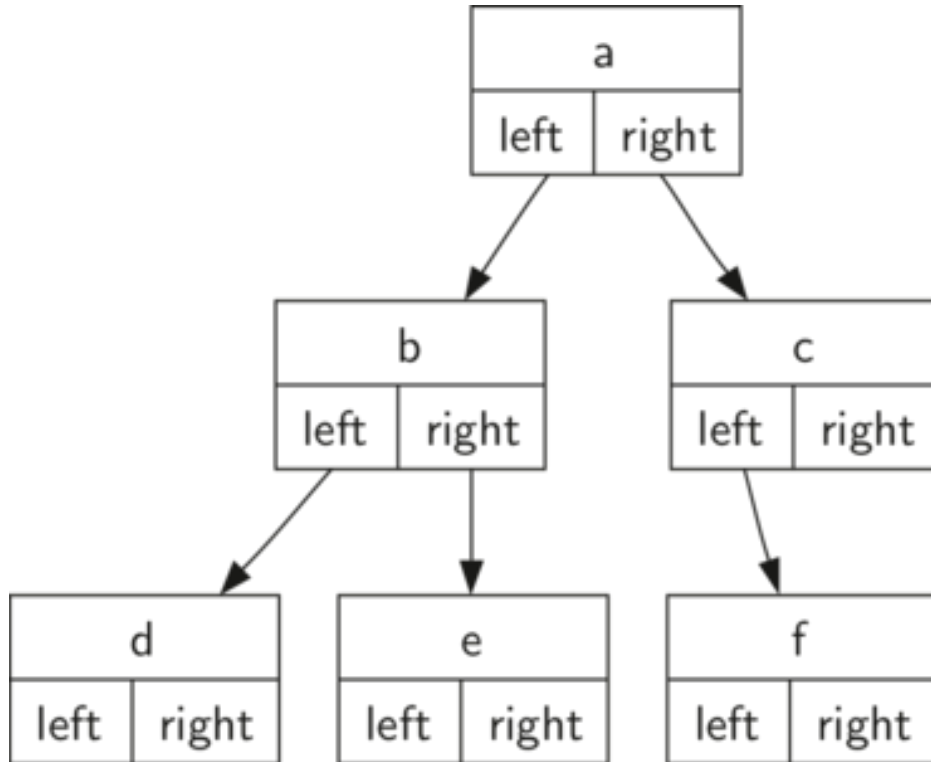
$i = 1$ $[0, 9, 2, 6, 5, 3]$

$i = 0$ $[0, 2, 3, 6, 5, 9]$

```
def buildHeap(self, alist):  
    i = len(alist) // 2  
    self.currentSize = len(alist)  
    self.heapList = [0] + alist[:]  
    while (i > 0):  
        self.percDown(i)  
        i = i - 1
```

ปมและการอ้างอิง
(nodes and references)

root และ subtree ซ้าย ขวา



```
class BinaryTree:
```

```
    def __init__(self, rootObj):
```

```
        self.key = rootObj
```

```
        self.leftChild = None
```

```
        self.rightChild = None
```

insertLeft และ insertRight

```
def insertLeft(self, newNode):  
    if self.leftChild == None:  
        self.leftChild = BinaryTree(newNode)  
    else:  
        t = BinaryTree(newNode)  
        t.leftChild = self.leftChild  
        self.leftChild = t
```

```
def insertRight(self, newNode):  
    if self.rightChild == None:  
        self.rightChild = BinaryTree(newNode)  
    else:  
        t = BinaryTree(newNode)  
        t.rightChild = self.rightChild  
        self.rightChild = t
```

อธิบายเป็นภาพการทำงานได้อย่างไร?

method อื่นๆ

```
def getRightChild(self):  
    return self.rightChild
```

```
def getLeftChild(self):  
    return self.leftChild
```

```
def setRootVal(self,obj):  
    self.key = obj
```

```
def getRootVal(self):  
    return self.key
```

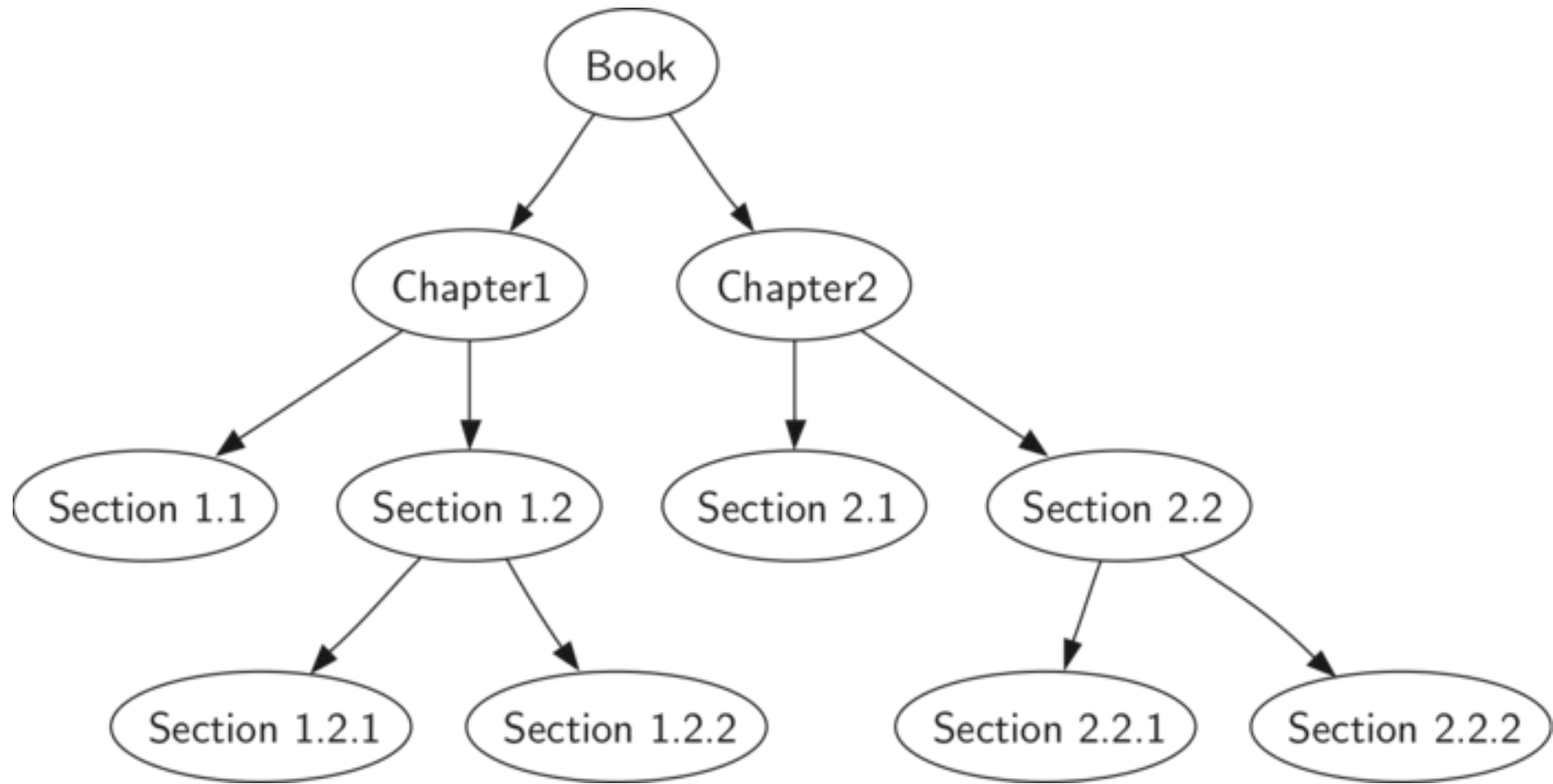
```
1 class BinaryTree:
2     def __init__(self, rootObj):
3         self.key = rootObj
4         self.leftChild = None
5         self.rightChild = None
6
7     def insertLeft(self, newNode):
8         if self.leftChild == None:
9             self.leftChild = BinaryTree(newNode)
10        else:
11            t = BinaryTree(newNode)
12            t.leftChild = self.leftChild
13            self.leftChild = t
14
15    def insertRight(self, newNode):
16        if self.rightChild == None:
17            self.rightChild = BinaryTree(newNode)
18        else:
19            t = BinaryTree(newNode)
20            t.rightChild = self.rightChild
21            self.rightChild = t
22
23
```

```
24 def getRightChild(self):
25     return self.rightChild
26
27 def getLeftChild(self):
28     return self.leftChild
29
30 def setRootVal(self, obj):
31     self.key = obj
32
33 def getRootVal(self):
34     return self.key
35
36
37 r = BinaryTree('a')
38 print(r.getRootVal())
39 print(r.getLeftChild())
40 r.insertLeft('b')
41 print(r.getLeftChild())
42 print(r.getLeftChild().getRootVal())
43 r.insertRight('c')
44 print(r.getRightChild())
45 print(r.getRightChild().getRootVal())
46 r.getRightChild().setRootVal('hello')
47 print(r.getRightChild().getRootVal())
48
```

การเดินทางในต้นไม้ (tree traversal)

การเดินทาง 3 แบบ

- preorder ลำดับก่อน
 - root $\rightarrow$ left subtree $\rightarrow$ right subtree
- postorder ลำดับหลัง
 - left subtree $\rightarrow$ right subtree $\rightarrow$ root
- inorder ในลำดับ
 - left subtree $\rightarrow$ root $\rightarrow$ right subtree



ควรจะเดินทางแบบไหน?

preorder

```
def preorder(tree):  
    if tree:  
        print(tree.getRootVal())  
        preorder(tree.getLeftChild())  
        preorder(tree.getRightChild())
```

แบบ function แยก

```
def preorder(self):  
    print(self.key)  
    if self.leftChild:  
        self.leftChild.preorder()  
    if self.rightChild:  
        self.rightChild.preorder()
```

แบบ method

```
def postorder(tree):
```

```
    if tree != None:
```

```
        postorder(tree.getLeftChild())
```

```
        postorder(tree.getRightChild())
```

```
    print(tree.getRootVal())
```

postorder

```
def postordereval(tree):
```

```
   opers = {'+':operator.add, '-':operator.sub, '*':operator.mul, '/':operator.truediv}
```

```
    res1 = None
```

```
    res2 = None
```

```
    if tree:
```

```
        res1 = postordereval(tree.getLeftChild())
```

```
        res2 = postordereval(tree.getRightChild())
```

```
        if res1 and res2:
```

```
            return opers[tree.getRootVal()](res1,res2)
```

```
        else:
```

```
            return tree.getRootVal()
```

inorder

```
def inorder(tree):  
    if tree != None:  
        inorder(tree.getLeftChild())  
        print(tree.getRootVal())  
        inorder(tree.getRightChild())
```

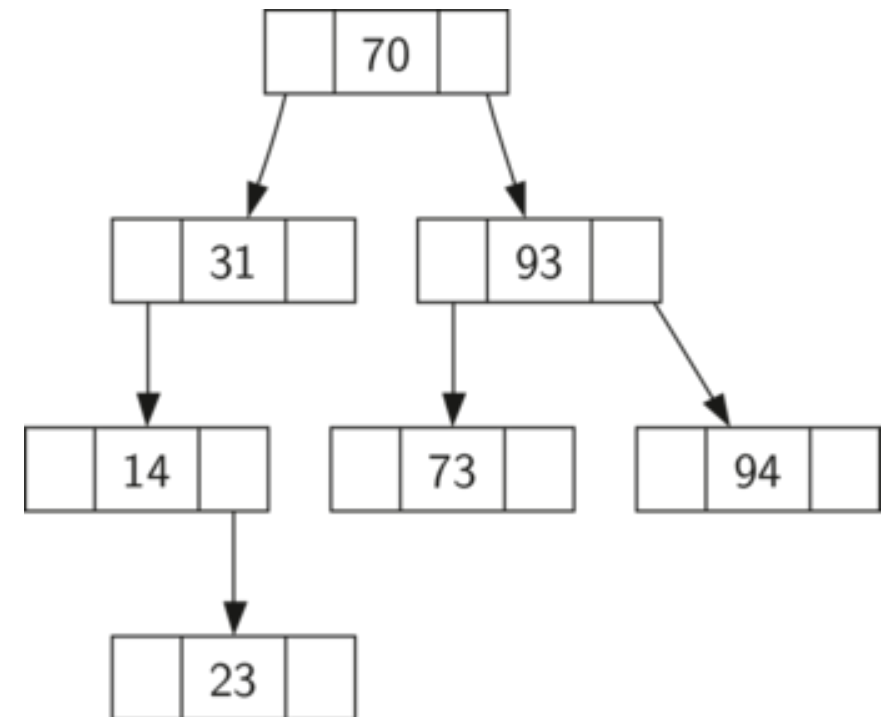
```
def printexp(tree):  
    sVal = ""  
    if tree:  
        sVal = '(' + printexp(tree.getLeftChild())  
        sVal = sVal + str(tree.getRootVal())  
        sVal = sVal + printexp(tree.getRightChild())+')'  
    return sVal
```

ต้นไม้ค้นหาแบบทวิภาค (binary search tree)

คุณสมบัติ

- ทุก **key** ของ **left subtree** จะต้องมิต่ำน้อยกว่า **key** ของ **root** เสมอ
- ทุก **key** ของ **right subtree** จะต้องมิต่ำมากกว่า **key** ของ **root** เสมอ
- เราจะสร้าง **search tree** สำหรับ **map**

70,31,93,94,14,23,73



```
class BinarySearchTree:
```

```
    def __init__(self):  
        self.root = None  
        self.size = 0
```

```
    def length(self):  
        return self.size
```

```
    def __len__(self):  
        return self.size
```

```
class TreeNode:
```

```
    def __init__(self, key, val, left=None, right=None,  
                  parent=None):
```

```
        self.key = key  
        self.payload = val  
        self.leftChild = left  
        self.rightChild = right  
        self.parent = parent
```

```
    def hasLeftChild(self):  
        return self.leftChild
```

```
    def hasRightChild(self):  
        return self.rightChild
```

```
    def isLeftChild(self):  
        return self.parent and self.parent.leftChild == self
```

```
    def isRightChild(self):  
        return self.parent and self.parent.rightChild == self
```

```
    def isRoot(self):
```

```
        return not self.parent
```

```
    def isLeaf(self):
```

```
        return not (self.rightChild or self.leftChild)
```

```
    def hasAnyChildren(self):
```

```
        return self.rightChild or self.leftChild
```

```
    def hasBothChildren(self):
```

```
        return self.rightChild and self.leftChild
```

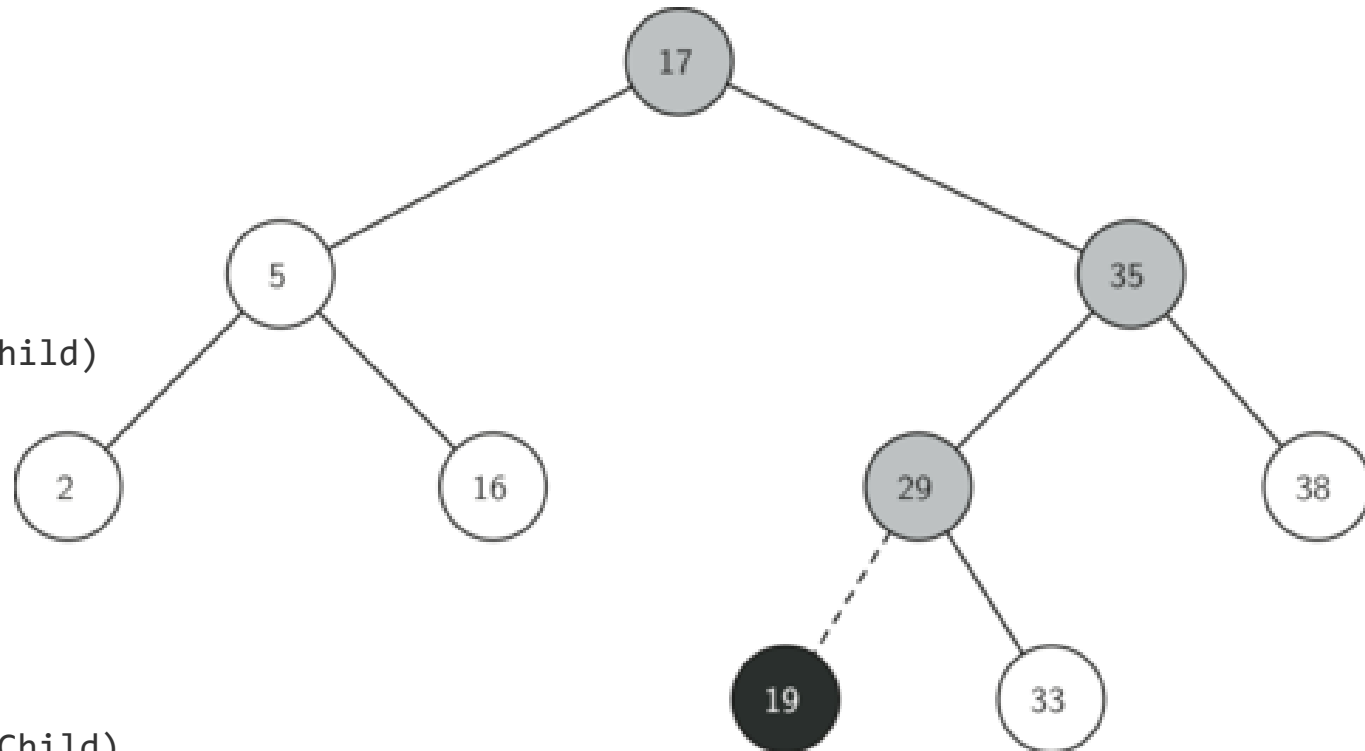
```
    def replaceNodeData(self, key, value, lc, rc):
```

```
        self.key = key  
        self.payload = value  
        self.leftChild = lc  
        self.rightChild = rc  
        if self.hasLeftChild():  
            self.leftChild.parent = self  
        if self.hasRightChild():  
            self.rightChild.parent = self
```

```
def put(self, key, val):
    if self.root:
        self._put(key, val, self.root)
    else:
        self.root = TreeNode(key, val)
    self.size = self.size + 1
```

```
def _put(self, key, val, currentNode):
    if key < currentNode.key:
        if currentNode.hasLeftChild():
            self._put(key, val, currentNode.leftChild)
        else:
            currentNode.leftChild =
TreeNode(key, val, parent=currentNode)
    else:
        if currentNode.hasRightChild():
            self._put(key, val, currentNode.rightChild)
        else:
            currentNode.rightChild =
TreeNode(key, val, parent=currentNode)
```

```
def __setitem__(self, k, v):
    self.put(k, v)
```



```
def get(self, key):
    if self.root:
        res = self._get(key, self.root)
        if res:
            return res.payload
        else:
            return None
    else:
        return None

def _get(self, key, currentNode):
    if not currentNode:
        return None
    elif currentNode.key == key:
        return currentNode
    elif key < currentNode.key:
        return self._get(key, currentNode.leftChild)
    else:
        return self._get(key, currentNode.rightChild)
```

```
def __getitem__(self, key):
    return self.get(key)

def __contains__(self, key):
    if self._get(key, self.root):
        return True
    else:
        return False

if 'Northfield' in myZipTree:
    print("oom ya ya")
```

```

def delete(self, key):
    if self.size > 1:
        nodeToRemove = self._get(key, self.root)
        if nodeToRemove:
            self.remove(nodeToRemove)
            self.size = self.size - 1
        else:
            raise KeyError('Error, key not in tree')
    elif self.size == 1 and self.root.key == key:
        self.root = None
        self.size = self.size - 1
    else:
        raise KeyError('Error, key not in tree')

def __delitem__(self, key):
    self.delete(key)

```

- remove มี 3 กรณี
 - node ที่ต้องการ remove ไม่มีลูก
 - node ที่ต้องการ remove มีลูก 1 node
 - node ที่ต้องการ remove มีลูก 2 nodes

```
if currentNode.isLeaf():
```

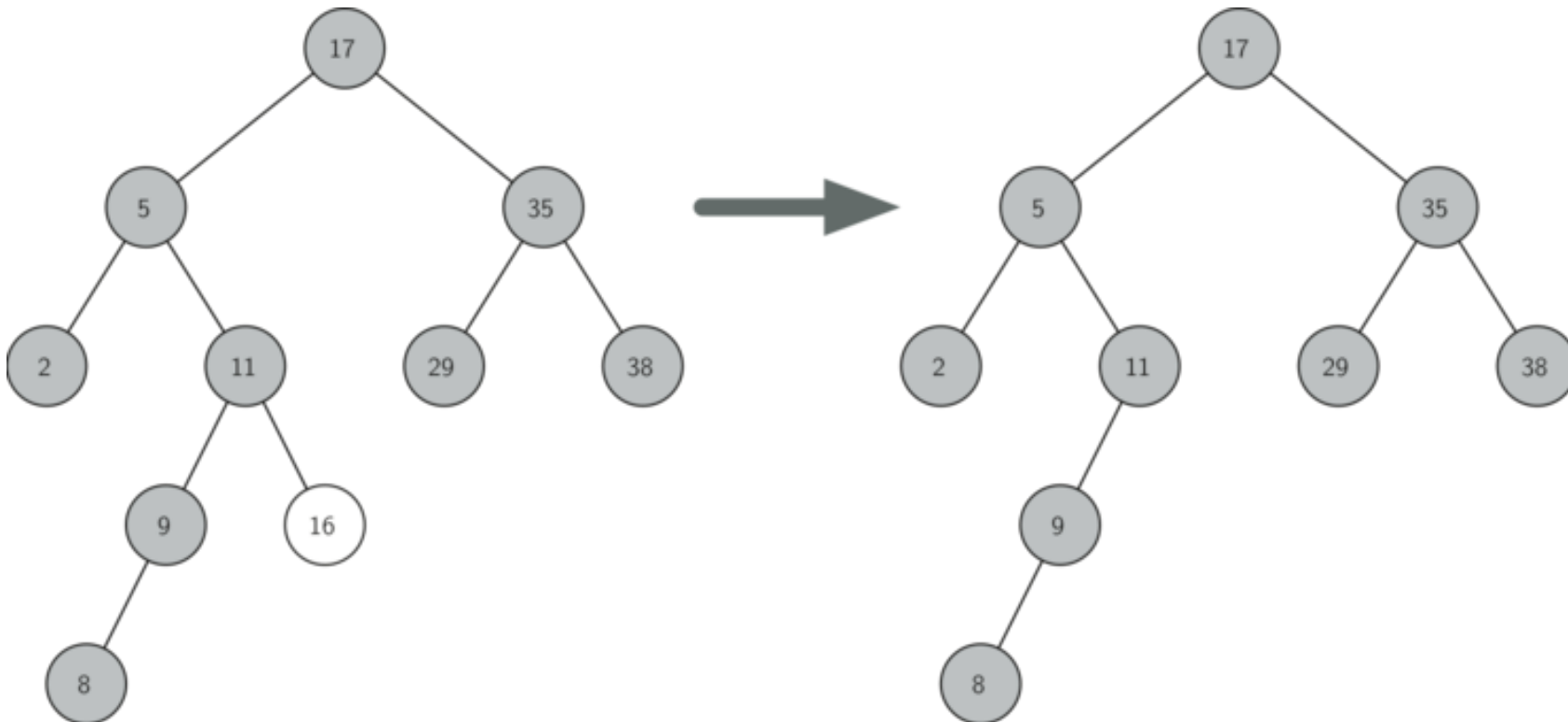
```
    if currentNode == currentNode.parent.leftChild:
```

```
        currentNode.parent.leftChild = None
```

```
    else:
```

```
        currentNode.parent.rightChild = None
```

กรณีเป็น leaf node



else: *# this node has one child*

if currentNode.hasLeftChild():

if currentNode.isLeftChild():

currentNode.leftChild.parent = currentNode.parent

currentNode.parent.leftChild = currentNode.leftChild

elif currentNode.isRightChild():

currentNode.leftChild.parent = currentNode.parent

currentNode.parent.rightChild = currentNode.leftChild

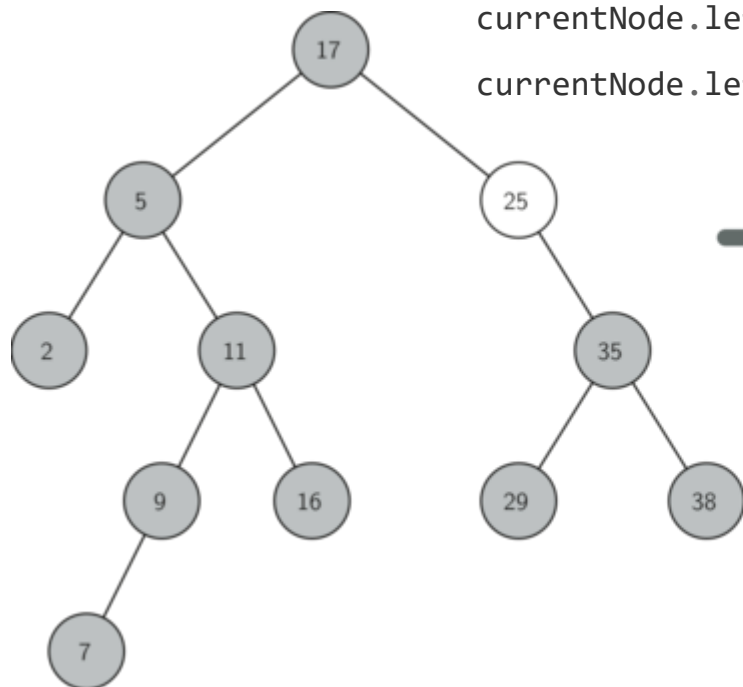
else:

currentNode.replaceNodeData(currentNode.leftChild.key,

currentNode.leftChild.payload,

currentNode.leftChild.leftChild,

currentNode.leftChild.rightChild)



else:

if currentNode.isLeftChild():

currentNode.rightChild.parent = currentNode.parent

currentNode.parent.leftChild = currentNode.rightChild

elif currentNode.isRightChild():

currentNode.rightChild.parent = currentNode.parent

currentNode.parent.rightChild = currentNode.rightChild

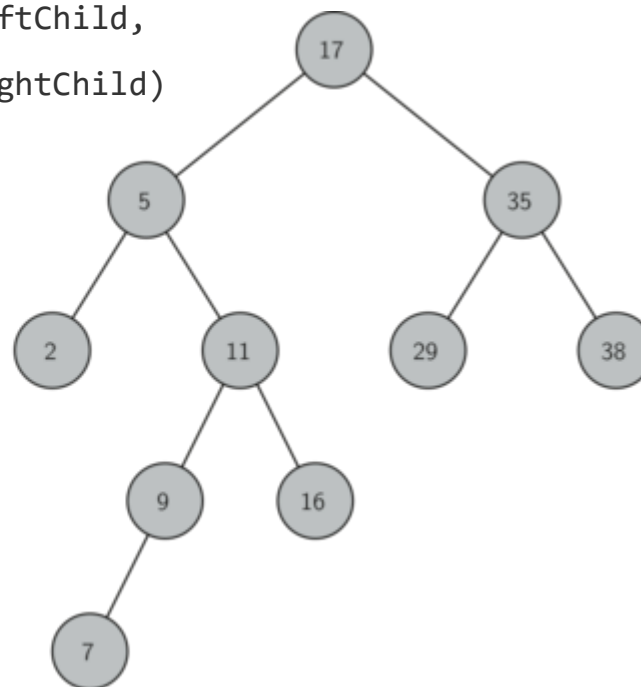
else:

currentNode.replaceNodeData(currentNode.rightChild.key,

currentNode.rightChild.payload,

currentNode.rightChild.leftChild,

currentNode.rightChild.rightChild)



กรณีมีลูก **1** ให้เอาลูกมาแทน
ตำแหน่งตัวเอง

กรณีมีลูก 2

- หาตัวแทนที่ยังรักษาคุณสมบัติของ **search tree**
 - ทุก **key** ของ **left subtree** จะต้องน้อยกว่า **key** ของ **root** เสมอ
 - ทุก **key** ของ **right subtree** จะต้องมากกว่า **key** ของ **root** เสมอ
- ตัวแทนคือ **node** ที่มี **key** มากเป็นลำดับถัดไปจาก **key** ของ **node** ที่ต้องการ **remove** (**successor**)
- ตัวแทนจะมีลูกไม่เกิน **1** ตัว
 - เก็บตัวแทนไว้
 - **remove** ตัวแทนที่ตำแหน่งเก่า
 - ใส่ตัวแทนกลับเข้าไปที่ตำแหน่ง **node** ที่ต้องการ **remove**

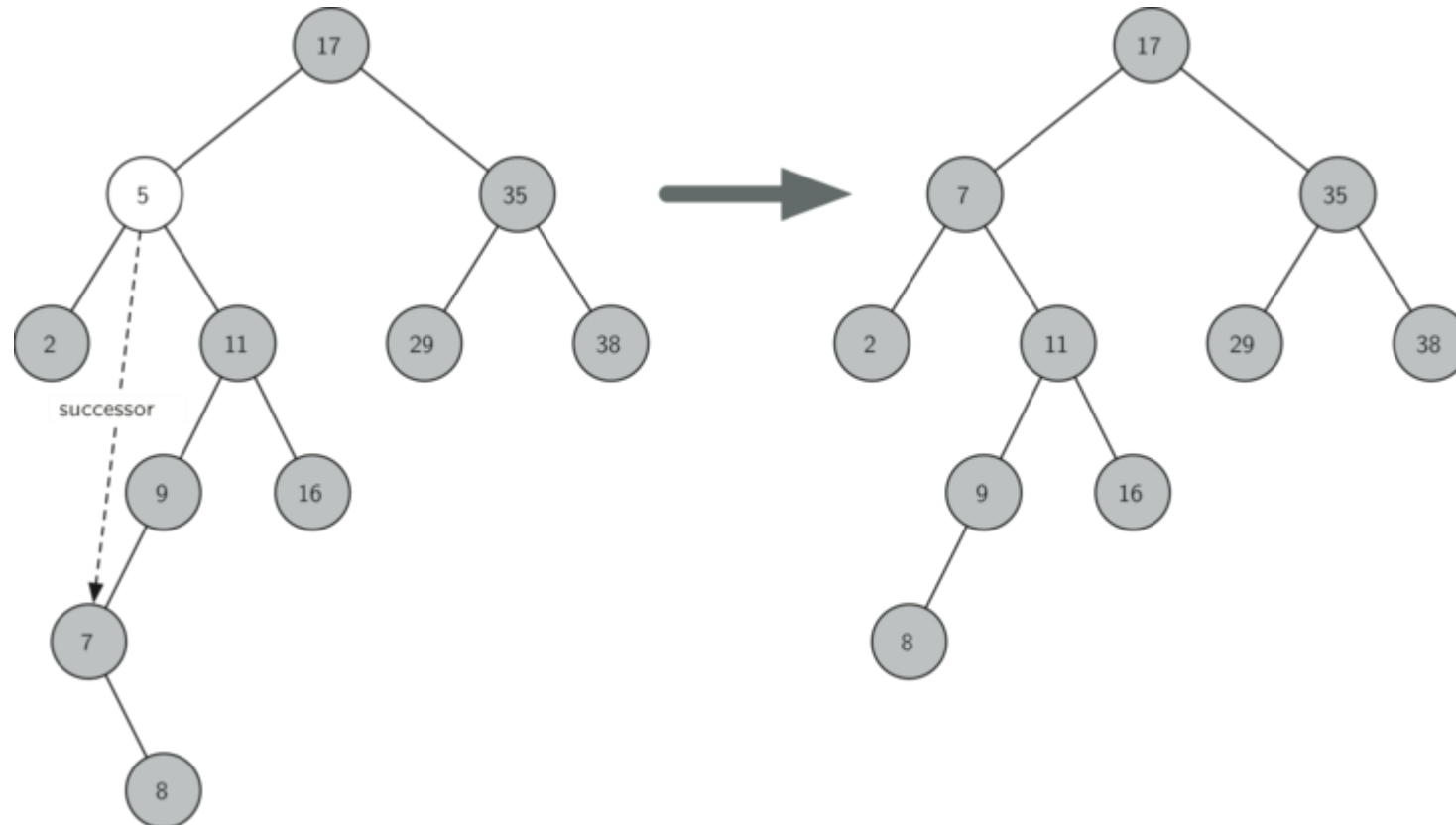
```
elif currentNode.hasBothChildren(): #interior
```

```
    succ = currentNode.findSuccessor()
```

```
    succ.spliceOut()
```

```
    currentNode.key = succ.key
```

```
    currentNode.payload = succ.payload
```



findSuccessor()

- พิจารณาในกรณีทั่วไป
- ถ้ามี **right child** แล้ว **successor** คือ **node** ที่มีค่า **key** น้อยสุดของ **right subtree**
- ถ้าไม่มี **right child** และตัวมันเองเป็น **left child** แล้ว **successor** คือ **parent**
- ถ้าไม่มี **right child** และตัวมันเองเป็น **right child** แล้ว **successor** คือ **successor** ของ **parent** ที่ไม่รวมตัวมันเอง

```
def findSuccessor(self):
    succ = None
    if self.hasRightChild():
        succ = self.rightChild.findMin()
    else:
        if self.parent:
            if self.isLeftChild():
                succ = self.parent
            else:
                self.parent.rightChild = None
                succ = self.parent.findSuccessor()
                self.parent.rightChild = self
    return succ

def findMin(self):
    current = self
    while current.hasLeftChild():
        current = current.leftChild
    return current
```

```
def spliceOut(self):
    if self.isLeaf():
        if self.isLeftChild():
            self.parent.leftChild = None
        else:
            self.parent.rightChild = None
    elif self.hasAnyChildren():
        if self.hasLeftChild():
            if self.isLeftChild():
                self.parent.leftChild = self.leftChild
            else:
                self.parent.rightChild = self.leftChild
                self.leftChild.parent = self.parent
        else:
            if self.isLeftChild():
                self.parent.leftChild = self.rightChild
            else:
                self.parent.rightChild = self.rightChild
            self.rightChild.parent = self.parent
```

ต้นไม้เอวีแอล

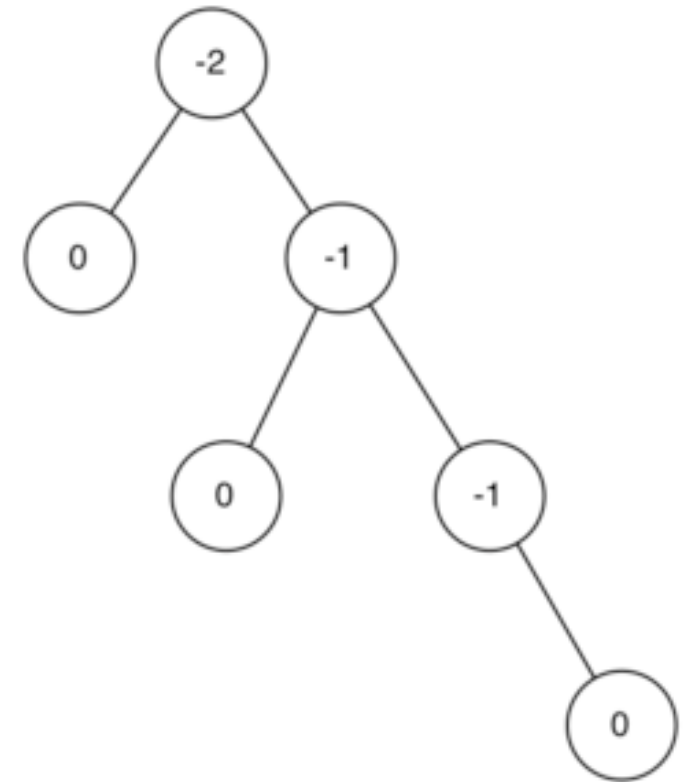
AVL Tree

ADT

- มี **method** เหมือนกับ **binary search tree**
- ต่างกันที่การทำงานข้างใน **method**
 - คำนึงถึงค่าความสมดุล (**balance factor**)

Balance factor

- $\text{balanceFactor} = \text{height}(\text{leftSubTree}) - \text{height}(\text{rightSubTree})$
- balance factor > 0 ต้นไม้หนักไปทางฝั่งซ้าย
- balance factor < 0 ต้นไม้หนักไปทางฝั่งขวา
- balance factor $= 0$ ต้นไม้สมดุลแบบสมบูรณ์
- AVL tree
 - balance factor = -1, 0, 1
 - ถ้าหลุดออกจากค่าพวกนี้ต้องทำการแก้ไข
 - ทำงานได้เร็วขึ้น



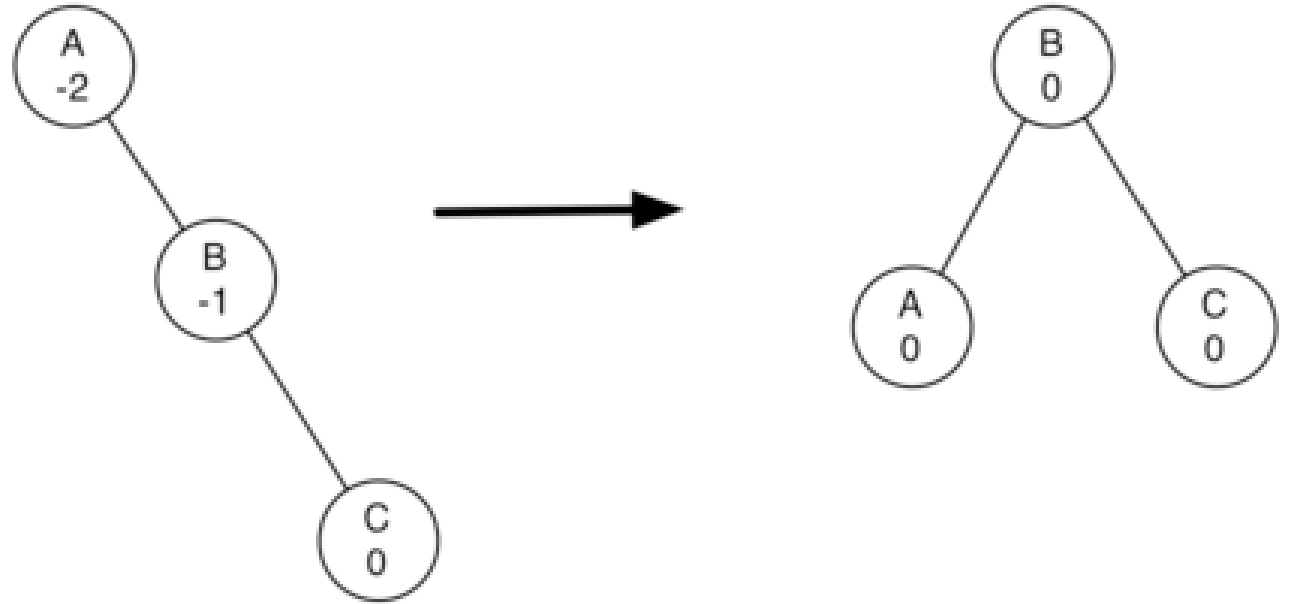
เพิ่ม node

- เพิ่ม left node
 - parent และ ต้นตระกูลมีค่า balance factor เพิ่ม 1
- เพิ่ม right node
 - parent และ ต้นตระกูลมีค่า balance factor ลด 1

```
def _put(self, key, val, currentNode):  
    if key < currentNode.key:  
        if currentNode.hasLeftChild():  
            self._put(key, val, currentNode.leftChild)  
        else:  
            currentNode.leftChild = TreeNode(key, val, parent=currentNode)  
            self.updateBalance(currentNode.leftChild)  
    else:  
        if currentNode.hasRightChild():  
            self._put(key, val, currentNode.rightChild)  
        else:  
            currentNode.rightChild = TreeNode(key, val, parent=currentNode)  
            self.updateBalance(currentNode.rightChild)
```

```
def updateBalance(self,node):  
    if node.balanceFactor > 1 or node.balanceFactor < -1:  
        self.rebalance(node)  
        return  
    if node.parent != None:  
        if node.isLeftChild():  
            node.parent.balanceFactor += 1  
        elif node.isRightChild():  
            node.parent.balanceFactor -= 1  
  
        if node.parent.balanceFactor != 0:  
            self.updateBalance(node.parent)
```

rebalance



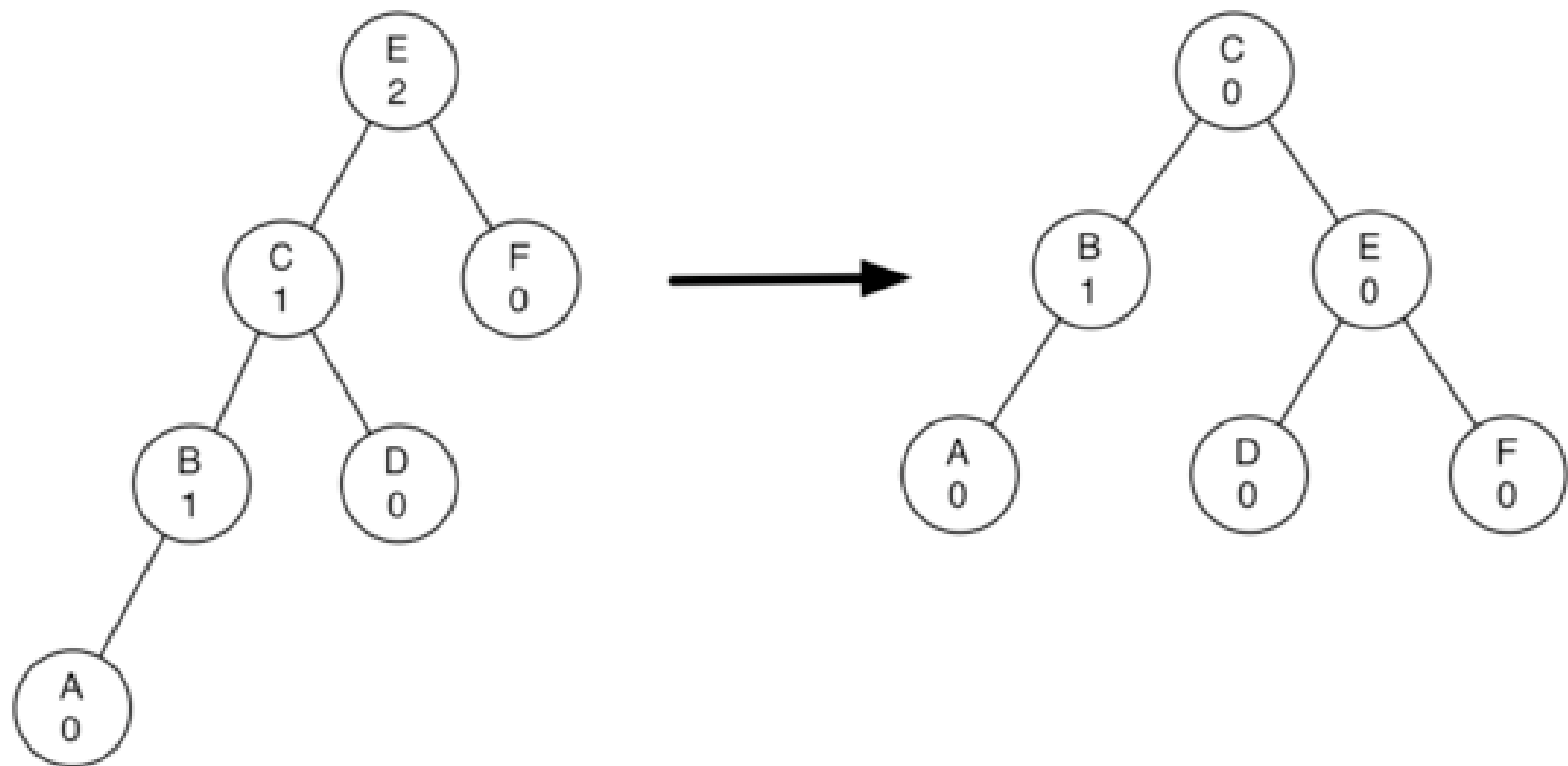
- rotation

- left rotation

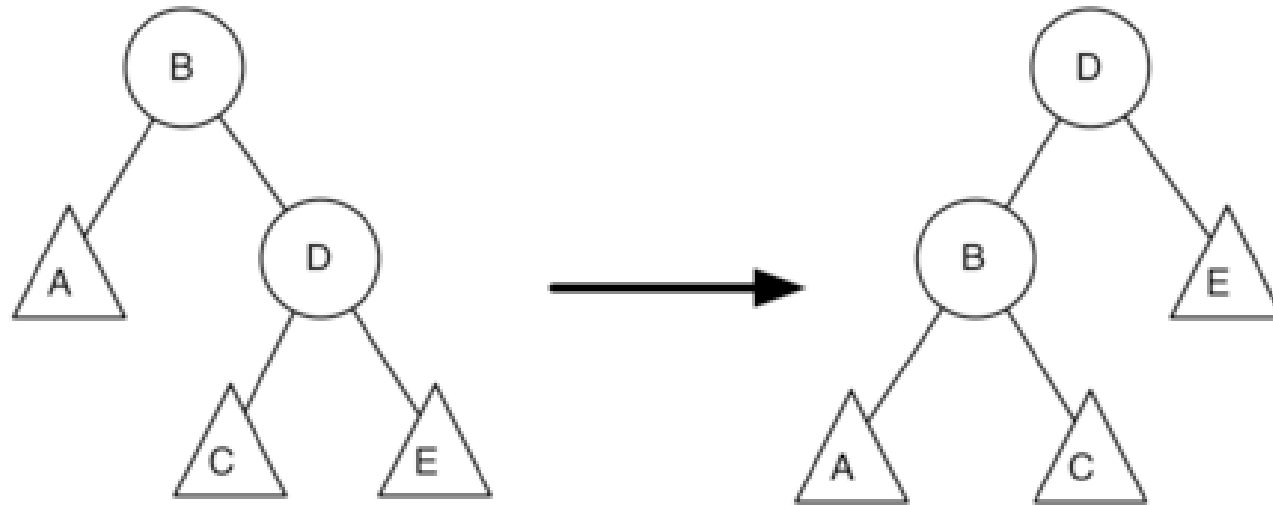
- ปรับ **right child** มาเป็น **root** ของ **subtree**
- เอา **root** เดิมมาเป็น **left child**
- ถ้า **root** ใหม่มี **left child** อยู่แล้วให้ปรับมาเป็น **right child** ของ **left child** ใหม่

- right rotation

- ปรับ **left child** มาเป็น **root** ของ **subtree**
- เอา **root** เดิมมาเป็น **right child**
- ถ้า **root** ใหม่มี **right child** อยู่แล้วให้ปรับมาเป็น **left child** ของ **right child** ใหม่



```
def rotateLeft(self, rotRoot):
    newRoot = rotRoot.rightChild
    rotRoot.rightChild = newRoot.leftChild
    if newRoot.leftChild != None:
        newRoot.leftChild.parent = rotRoot
    newRoot.parent = rotRoot.parent
    if rotRoot.isRoot():
        self.root = newRoot
    else:
        if rotRoot.isLeftChild():
            rotRoot.parent.leftChild = newRoot
        else:
            rotRoot.parent.rightChild = newRoot
    newRoot.leftChild = rotRoot
    rotRoot.parent = newRoot
    rotRoot.balanceFactor = rotRoot.balanceFactor + 1 - min(newRoot.balanceFactor, 0)
    newRoot.balanceFactor = newRoot.balanceFactor + 1 + max(rotRoot.balanceFactor, 0)
```



$$newBal(B) = h_A - h_C$$

$$oldBal(B) = h_A - h_D$$

$$oldBal(B) = h_A - (1 + \max(h_C, h_E))$$



$$newBal(B) - oldBal(B) = h_A - h_C - (h_A - (1 + \max(h_C, h_E)))$$

$$newBal(B) - oldBal(B) = h_A - h_C - h_A + (1 + \max(h_C, h_E))$$

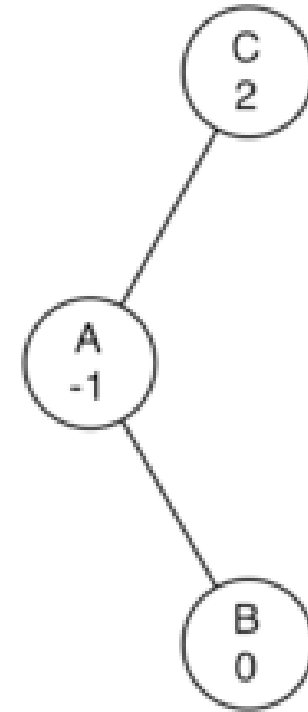
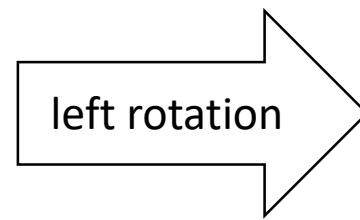
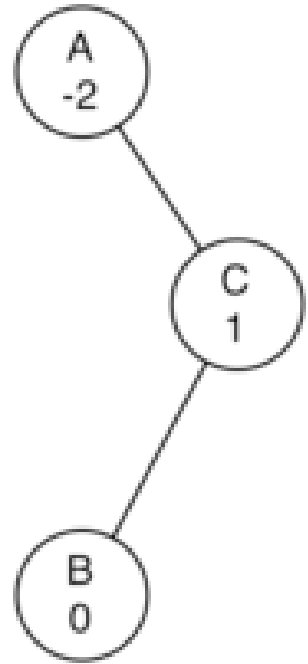
$$newBal(B) - oldBal(B) = h_A - h_A + 1 + \max(h_C, h_E) - h_C$$

$$newBal(B) - oldBal(B) = 1 + \max(h_C, h_E) - h_C$$

$$newBal(B) = oldBal(B) + 1 + \max(h_C - h_C, h_E - h_C)$$

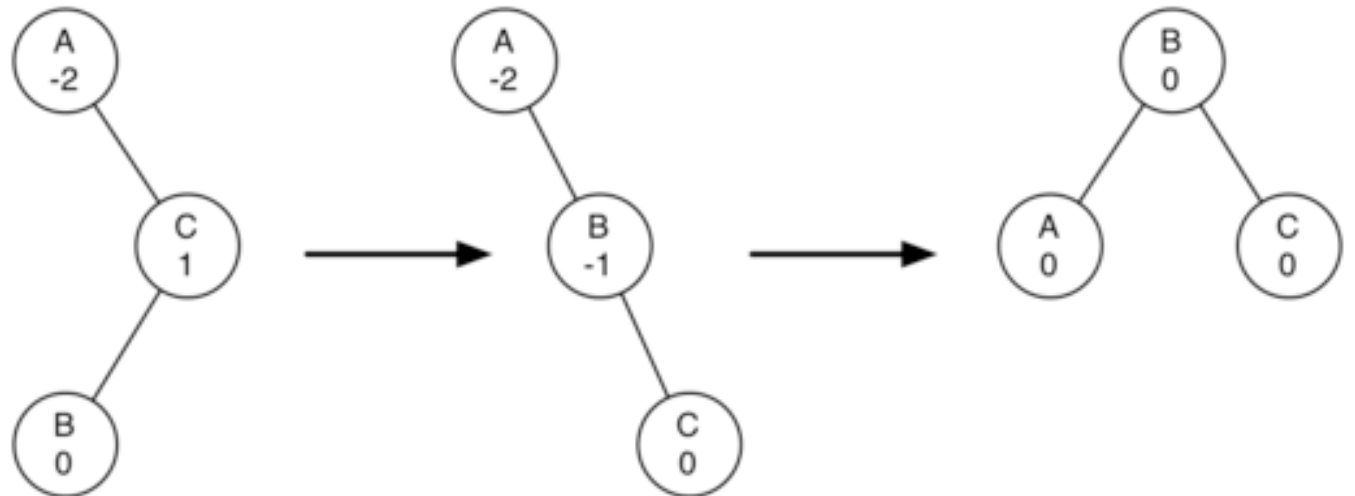
$$newBal(B) = oldBal(B) + 1 + \max(0, -oldBal(D))$$

$$newBal(B) = oldBal(B) + 1 - \min(0, oldBal(D))$$



double rotation

- ทำ left rotation
 - ถ้า right child มี balance factor > 0 (หนักไปทางฝั่งซ้าย) ให้ทำ right rotation ที่ right child ก่อน แล้วจึงทำ left rotation ปกติ
- ทำ right rotation
 - ถ้า left child มี balance factor < 0 (หนักไปทางฝั่งขวา) ให้ทำ left rotation ที่ left child ก่อน แล้วจึงทำ right rotation ปกติ

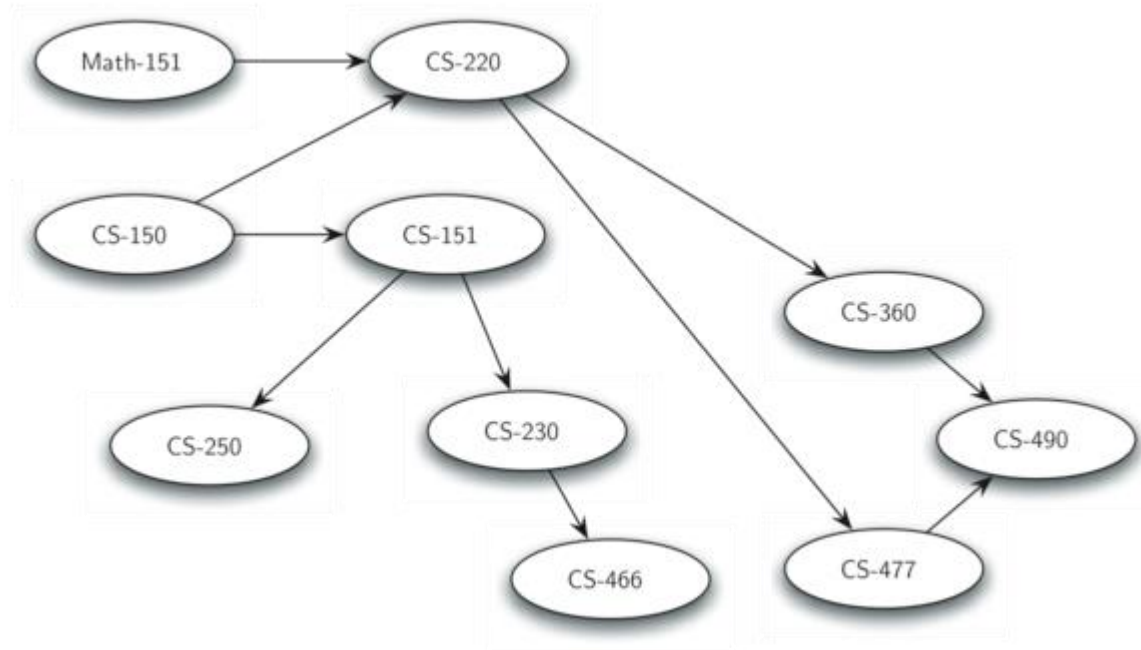


```
def rebalance(self,node):
    if node.balanceFactor < 0:
        if node.rightChild.balanceFactor > 0:
            self.rotateRight(node.rightChild)
            self.rotateLeft(node)
        else:
            self.rotateLeft(node)
    elif node.balanceFactor > 0:
        if node.leftChild.balanceFactor < 0:
            self.rotateLeft(node.leftChild)
            self.rotateRight(node)
        else:
            self.rotateRight(node)
```

กราฟ
graph

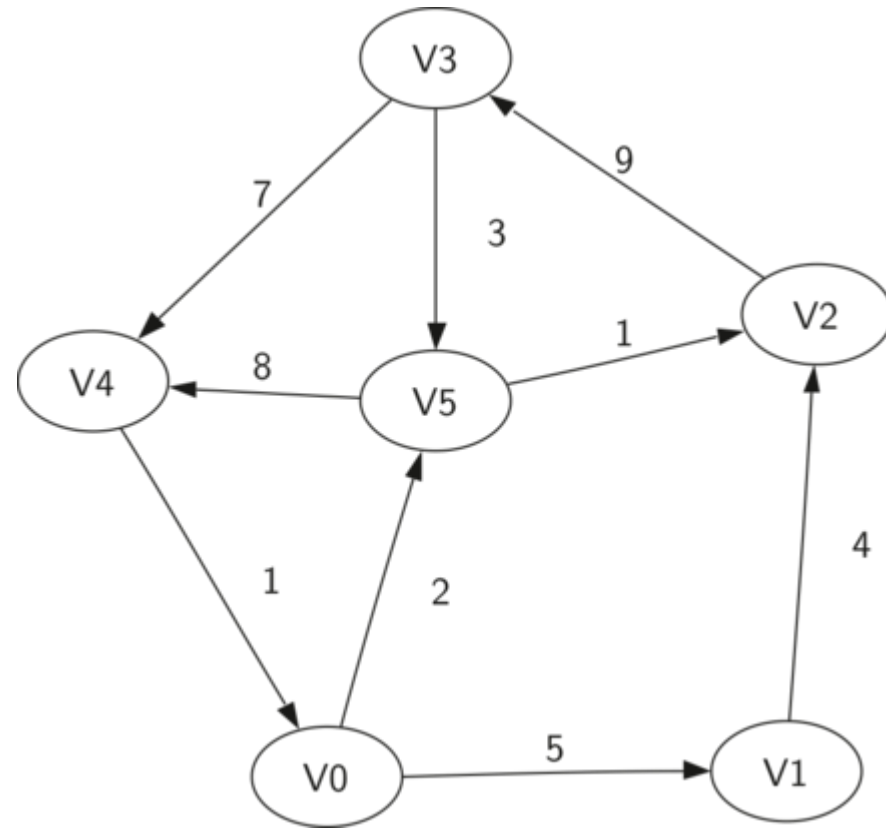
ใช้ทำอะไร?

- แสดงความสัมพันธ์ของข้อมูล
- อธิบายได้กว้างกว่าต้นไม้



คำศัพท์

- จุดยอด (vertex, node)
 - key
 - payload
- เส้นเชื่อม (edge, arc)
 - มีทิศทาง
 - ไม่มีทิศทาง
- น้ำหนัก (weight)
 - ระยะทาง
- $G = (V, E)$

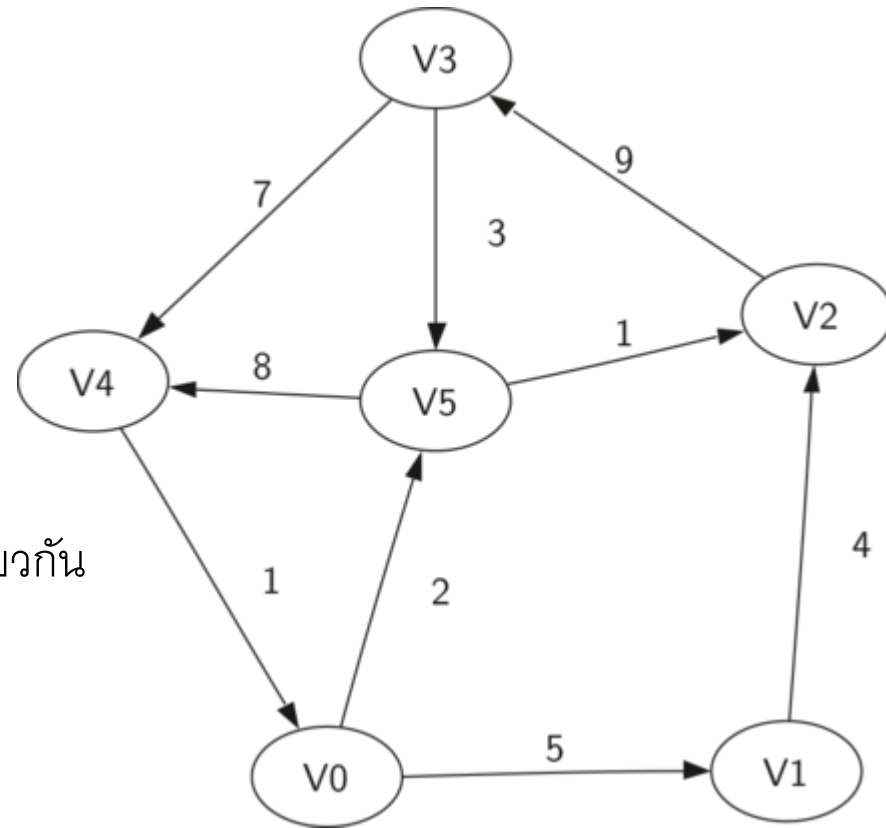


$V = \{V0, V1, V2, V3, V4, V5\}$

$E = \{(v0, v1, 5), (v1, v2, 4), (v2, v3, 9), (v3, v4, 7), (v4, v0, 1), (v0, v5, 2), (v5, v4, 8), (v3, v5, 3), (v5, v2, 1)\}$

คำศัพท์

- เส้นทาง (path)
 - ลำดับของจุดยอดที่เชื่อมโยงกัน
 - ความยาวของเส้นทางคำนวณจากน้ำหนัก
- วงวน (cycle)
 - จุดยอดเริ่มต้นและจุดยอดสุดท้ายของเส้นทางเป็นจุดเดียวกัน



(V3,V4,V0,V1)

{(v3,v4,7),(v4,v0,1),(v0,v1,5)}

graph ADT

- Graph() สร้าง graph ว่าง.
- addVertex(vert) เพิ่ม vertex ให้กับ graph.
- addEdge(fromVert, toVert) กำหนด edge ที่มีทิศทางให้กับ 2 vertex
- addEdge(fromVert, toVert, weight) กำหนด edge ที่มีทิศทางและน้ำหนักให้กับ 2 vertex
- getVertex(vertKey) ค้นหา vertex ด้วย key
- getVertices() คืนค่า list ของ vertex ทั้งหมดใน graph
- in คืนค่า True เมื่อมี vertex ที่ถามอยู่ใน graph คืนค่า False เมื่อไม่มี

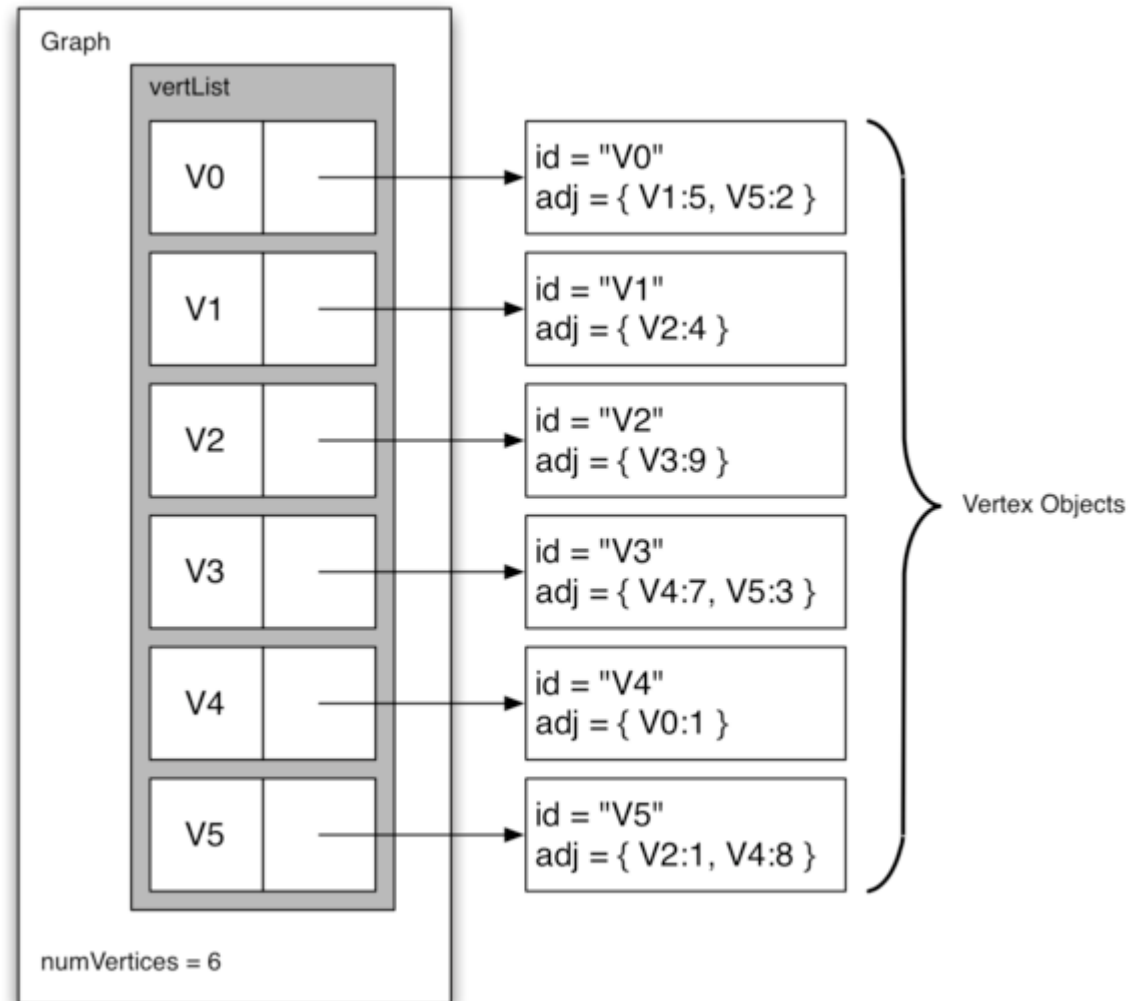
เมตริกซ์การเชื่อมโยง (adjacency matrix)

- ใช้ตารางแสดงความเชื่อมโยง
 - array 2 มิติ
- เห็นความเชื่อมโยงได้ง่าย
- เปลืองเนื้อที่
- ดีเมื่อมี **edge** เยอะ

	V0	V1	V2	V3	V4	V5
V0		5				2
V1			4			
V2				9		
V3					7	3
V4	1					
V5			1		8	

ลิสต์การเชื่อมโยง (adjacency list)

- กะทัดรัด
- หาความเชื่อมโยงได้ง่ายกว่า



```
class Vertex:
    def __init__(self, key):
        self.id = key
        self.connectedTo = {}

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight

    def __str__(self):
        return str(self.id) + ' connectedTo: ' + str([x.id for x in self.connectedTo])

    def getConnections(self):
        return self.connectedTo.keys()

    def getId(self):
        return self.id

    def getWeight(self, nbr):
        return self.connectedTo[nbr]
```

```
class Graph:
```

```
    def __init__(self):
```

```
        self.vertList = {}
```

```
        self.numVertices = 0
```

```
    def addVertex(self, key):
```

```
        self.numVertices = self.numVertices + 1
```

```
        newVertex = Vertex(key)
```

```
        self.vertList[key] = newVertex
```

```
        return newVertex
```

```
    def getVertex(self, n):
```

```
        if n in self.vertList:
```

```
            return self.vertList[n]
```

```
        else:
```

```
            return None
```

```
    def __contains__(self, n):
```

```
        return n in self.vertList
```

```
    def addEdge(self, f, t, cost=0):
```

```
        if f not in self.vertList:
```

```
            nv = self.addVertex(f)
```

```
        if t not in self.vertList:
```

```
            nv = self.addVertex(t)
```

```
        self.vertList[f].addNeighbor(self.vertList[t], cost)
```

```
    def getVertices(self):
```

```
        return self.vertList.keys()
```

```
    def __iter__(self):
```

```
        return iter(self.vertList.values())
```

```

>>> g = Graph()
>>> for i in range(6):
...     g.addVertex(i)
>>> g.vertList
{0: <adjGraph.Vertex instance at 0x41e18>,
 1: <adjGraph.Vertex instance at 0x7f2b0>,
 2: <adjGraph.Vertex instance at 0x7f288>,
 3: <adjGraph.Vertex instance at 0x7f350>,
 4: <adjGraph.Vertex instance at 0x7f328>,
 5: <adjGraph.Vertex instance at 0x7f300>}
>>> g.addEdge(0,1,5)
>>> g.addEdge(0,5,2)
>>> g.addEdge(1,2,4)
>>> g.addEdge(2,3,9)
>>> g.addEdge(3,4,7)
>>> g.addEdge(3,5,3)
>>> g.addEdge(4,0,1)
>>> g.addEdge(5,4,8)
>>> g.addEdge(5,2,1)

```

```

>>> for v in g:
...     for w in v.getConnections():
...         print("( %s , %s )" % (v.getId(), w.getId()))
...
( 0 , 5 )
( 0 , 1 )
( 1 , 2 )
( 2 , 3 )
( 3 , 4 )
( 3 , 5 )
( 4 , 0 )
( 5 , 4 )
( 5 , 2 )

```